

Verification Continuum™

VC Apps Protocol Analyzer User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	6
Synopsys Statement on Inclusivity and Diversity	6
1. Introduction	8
Tool Flow	9
Integration Process Overview	10
2. Concepts	13
Protocol Objects and Object Types	13
Stacked Protocols or Abstraction Layers	14
Parent or Child Relationships	16
Hierarchical Object Types	17
Object Fields or Attributes	18
Predecessor or Successor Relationships	19
Channels	20
Recording Memory Operations	21
Low-Level Memory Data	21
Initializing a Memory Using a Pattern	23
Loading Memory Data From a File	24
Correlating Memory Transactions With Memory Actions	25
3. Integration Process	27
Data Recorded During Simulation	27
Protocol Characteristics	28
Protocol Analysis and Object Type Identification	29
4. Directory Structure and Files	35
5. Protocol Extension XML File	40

XML Basics	40
General Protocol Information	42
Supported Methodologies	43
Protocol or Methodology Help Resources	45
Object Types or Object Type Hierarchies	48
Object Type Fields or Attributes	53
Signal Interface Groups	55
6. Integrating a VIP Using VC Apps Protocol Analyzer	58
Integration Example	58
Integration Steps	60
Creating a Protocol Extension Definition	61
Instrumenting the SystemVerilog Code	64
Modifying the Build or Simulation Environment	70
Modifying the Protocol Analyzer Invocation Environment	70
Configuration Control Considerations	71
Important Rules for Creating Objects	72
A. Appendix A- VC Apps Protocol Analyzer SystemVerilog Writer API	74
Writer Macros or Defines	75
ENABLE_VERDI_PA_WRITER	75
VERDI_PA_WRITER_BV_MAX_WIDTH	75
General Writer Methods	76
new (constructor)	76
flush_writer	77
Interface Methods	77
add_if_paths	77
General Object Methods	77
create_object	77
end_object	78
Object Relationship Methods	79
set_object_parent	79
add_object_child	80
add_object_children	80

Contents

set_object_predecessor	81
add_object_successor	82
add_object_successors	83
Object Attribute Methods	85
set_object_attribute_value_bit	85
set_object_attribute_value_bit_vector	85
set_object_attribute_value_int	86
set_object_attribute_value_real	87
set_object_attribute_value_time	88
set_object_attribute_value_string	89
Memory Methods	90
mem_init	90
mem_init_range	90
mem_read	91
mem_write	92
mem_write_masked	92
mem_peek	93
mem_poke	93
mem_load_file	94
mem_file_data	94
mem_unload	95
mem_unload_all	95

Preface

This VC Apps Protocol Analyzer User Guide describes the overall VC Apps Protocol Analyzer integration process as well as the specific steps that are required to integrate a protocol VIP into the Verdi Protocol Analyzer environment.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary

Contents

language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

As designs incorporate more (and more complex) standard protocols, the engineers tasked with verifying the behavior of those designs find themselves faced with a variety of new challenges. Not only must they understand the domain-specific aspects of the design they are verifying, they must also understand the operation of the incorporated protocols and how those protocol interfaces interact with the design. Learning about a new protocol (or coming up to speed on updates to a well-established protocol) may represent significant effort for design verification teams. Large SOC's that incorporate multiple protocol blocks further exacerbate these issues.

The complexity of modern protocols also creates a significant challenge for many traditional debug methodologies.

Some methodologies utilize a *bottom-up* approach. That is, they extract information from simulation log or waveform dump files and then attempt to transform this low-level data into more easily analyzed higher-level representations. But this approach assumes not only that all of the required high-level information can be inferred from the low-level data but also that this approach is reasonably efficient.

Alternatively, the debug process may utilize a *top-down* approach. That is, this approach may be forced, particularly when a test environment focuses primarily on higher-level transactions. But top-down methodologies rarely offer adequate insight into the underlying simulation details, which are often required to uncover the ultimate cause of unexpected behavior.

The scope of complex protocol definitions is broad, extending from the lowest physical details (such as the analog behavior exhibited by a PHY) up to the edge of (and sometimes into) the software domain. This not only highlights the challenges faced by verification teams who wish to add *home-grown* support for a particular protocol to a verification environment, but also illustrates two important requirements for a protocol-oriented analysis tool. First, a protocol-oriented analysis tool must be able to represent and view protocol data at multiple levels of abstraction. Second, the tool must be extensible, such that these protocol-oriented analysis capabilities can be made available for in-house, internally-developed or proprietary protocol VIPs.

The *Verdi Protocol Analyzer* is designed to address these challenges. The *Verdi Protocol Analyzer* accelerates the investigation of protocol behavior by providing protocol-oriented analysis and debugging capabilities in verification environments that utilize Synopsys Verification IP. These capabilities not only reduce the time required to identify the source

of bugs that can be directly attributed to protocol behavior, but also provide insight into aspects of the operation of a design that might otherwise be overlooked. With the addition of *VC Apps Protocol Analyzer*, these same benefits can be extended to in-house, internally-developed or proprietary protocol VIPs.

Starting with the Verdi L-2016.06 release, the *VC Apps Protocol Analyzer APIs* were enhanced to support the recording of memory data of in-house, internally-developed or proprietary protocol VIPs. The use of these APIs allow you to visualize memory data using the *Verdi Memory Browser*, which is fully integrated with other transaction-based and general Verdi capabilities.

This document describes the overall *VC Apps Protocol Analyzer* integration process as well as the specific steps that are required to integrate a protocol VIP into the *Verdi Protocol Analyzer* environment.

Note:

Verdi also provides corresponding C and Tcl APIs to dump the protocol related transaction data from custom VIP into FSDB. For details, see *VC APPS Native Programming Interface (NPI) User Guide*.

Tool Flow

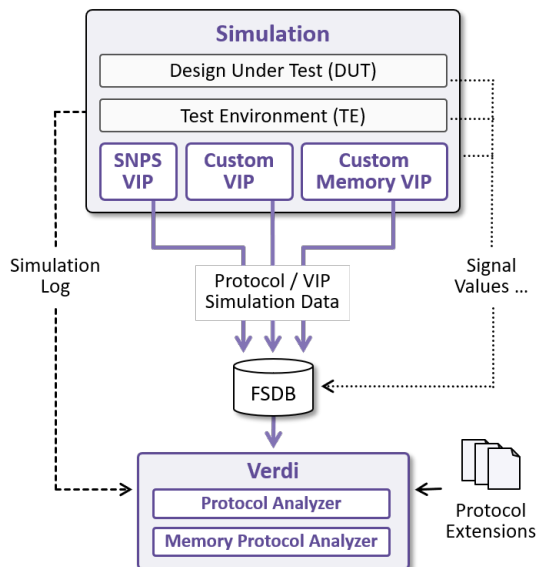
In the hardware world, bus analyzers and packet analyzers are widely used to debug hardware issues. These analyzers display information at a higher level of abstraction than more traditional logic analyzers that work at the signal level.

The *Verdi Protocol Analyzer* provides similar capabilities for verification engineers who are working with RTL and face many of the same debug challenges. The *Verdi Protocol Analyzer* brings all of the design and test environment debug information into one place and provides a *protocol-aware* environment to simplify root-cause analysis.

During simulation, the VIP captures protocol-specific data using code that is either designed into the Synopsys VIP or added to a in-house or custom VIP as part of the *VC Apps Protocol Analyzer* integration process. As the VIP interacts with the design, it creates a high-level record of transactions and other interesting protocol-specific activity (such as the values of internal state machines). This protocol-specific (and simulation-specific) data is stored in a common FSDB database along with the other design information, such as waveforms, source code, stack traces, and so on.

The *Verdi Protocol Analyzer* uses this data in conjunction with protocol knowledge provided by the Synopsys VIP (or created as part of the *VC Apps Protocol Analyzer* integration process) to create a higher-level, interactive display of the protocol-related activity and provide comprehensive protocol-oriented analysis and debug capabilities. The protocol-related activity can also be synchronized with other design-related debug information (such as signal activity and simulation log messages) to enable easy correlation of errors (or other unexpected behavior) with protocol activity.

Figure 1 System Topology With Integration of Custom VIPs



Integration Process Overview

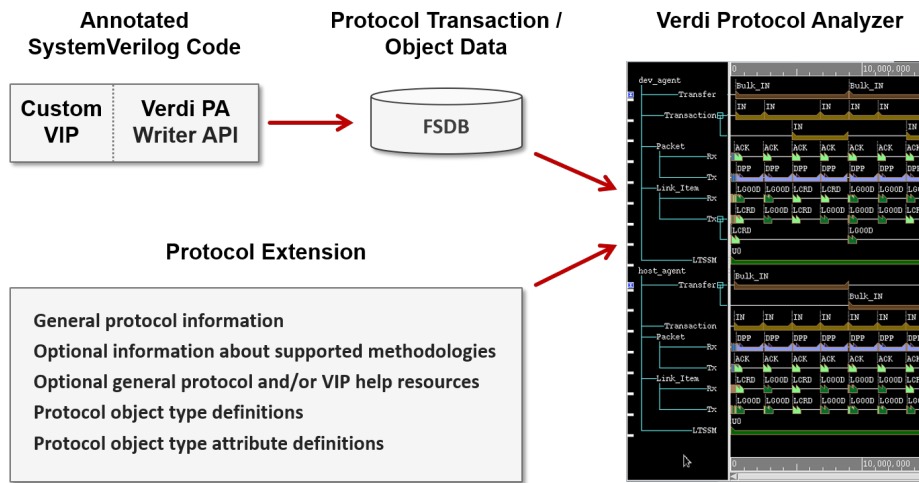
VC Apps Protocol Analyzer provides an open interface that enables integration of non-Synopsys VIPs (in-house or custom VIPs or other protocol-like components) into the Verdi environment so as to facilitate access to the comprehensive protocol-specific debugging capabilities provided by the Verdi platform.

In this flow, protocol-related data from these VIPs can be recorded into a common FSDB file. This data is then processed in conjunction with a protocol extension definition to enable the protocol-oriented debug and analysis capabilities of the *Verdi Protocol Analyzer*.

The *VC Apps Protocol Analyzer* integration process involves the following two steps:

1. In the first step, existing custom or in-house SystemVerilog VIP code is annotated with calls to the *VC Apps Protocol Analyzer SystemVerilog API*, which is defined in the `verdi_pa_writer` class that is delivered as part of the Verdi installation. This API enables the recording of protocol-related activity to an FSDB file during simulation.
2. In the second step, you can create a protocol extension definition, which describes the specific characteristics of the protocol information that was captured during simulation.

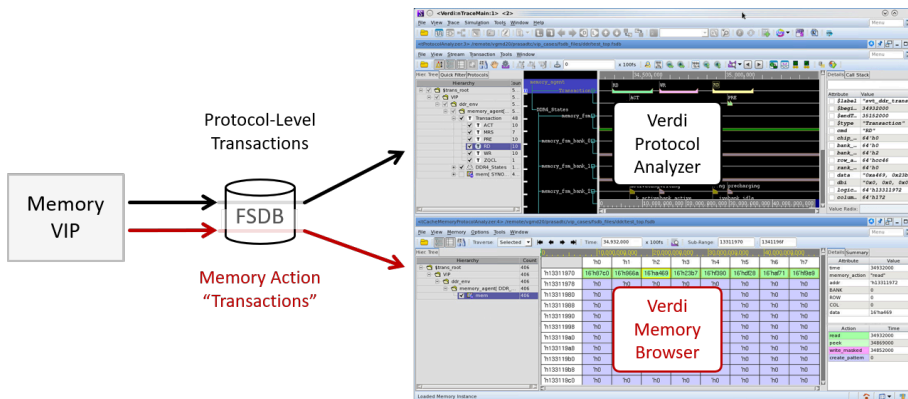
Figure 2 Overview of the VC Apps Protocol Analyzer Integration Process



General protocol information is captured as part of the protocol extension definition creation process. Note that this information is captured once for a particular protocol¹ as part of the integration process and is used by the *Verdi Protocol Analyzer* when processing data from any instance of that protocol that captures protocol-related data during a simulation run.

The *VC Apps Protocol Analyzer APIs* can be used to capture and view both high-level memory protocol transactions and low-level memory actions, as illustrated in the following figure:

Figure 3 Capturing and Viewing High-Level and Low-Level Memory Operations



1. More specifically, this information described in a protocol extension definition is captured for a particular version of a protocol. Protocol definitions can change over time. Also, the information that is captured by a VIP can change over time. These possibilities are supported by the *Verdi Protocol Analyzer* through versioning of the protocol extension definitions.

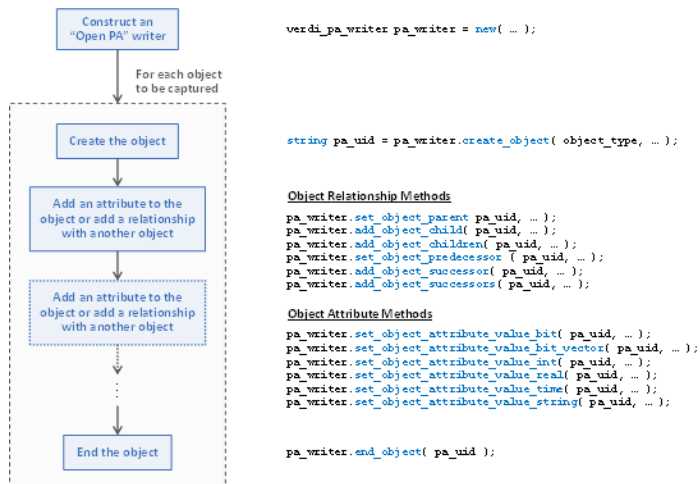
The process of annotating SystemVerilog VIP code with calls to the *VC Apps Protocol Analyzer SystemVerilog API* is usually a straight-forward process, once the kinds of objects of interest are identified. The analysis used to identify such objects is discussed in detail in the following section. This process is shown in [Overview of the SystemVerilog Code Annotation Flow](#).

The term *object* has appeared a number of times in this document, where the term *transaction* is expected. The term *object* is used to encourage a more broad analysis of the protocol when determining the information which needs to be described in the protocol extension definition. Also, to capture during simulation for viewing during the debug process.

Ideally, a VIP that captures protocol-related activity using *VC Apps Protocol Analyzer* creates a high-level record of not only transactions but other interesting protocol-specific activity (such as the values of internal state machines). Even with the use of the term *object*, all data that is captured using the *VC Apps Protocol Analyzer* is represented in the FSDB as *transactions* (and can also be viewed and analyzed using the general transaction-based capabilities of the Verdi platform such as *tBrowser*, *tAnalyzer*, and *tRelation*).

Finally, since the *VC Apps Protocol Analyzer* process supports versioning, the integration of a VIP might begin by first capturing the basic transactions and then enhanced to capture additional data in the later versions.

Figure 4 Overview of the SystemVerilog Code Annotation Flow



2

Concepts

This chapter discusses about the underlying concepts for *VC Apps Protocol Analyzer* in the following sections:

- [Protocol Objects and Object Types](#)
- [Stacked Protocols or Abstraction Layers](#)
- [Hierarchical Object Types](#)
- [Object Fields or Attributes](#)
- [Predecessor or Successor Relationships](#)
- [Channels](#)
- [Recording Memory Operations](#)

Protocol Objects and Object Types

Verdi Protocol Analyzer is a protocol-oriented debug and analysis environment. As such, the goal of the tool is to provide visualization and analysis of *protocol objects*. The term *protocol object* refers to any description of data that is found in a protocol specification.

For example, consider a protocol specification that defines a packet *object* as a particular sequence of bytes and defines a transfer *object* as a particular collection of packets. In the *Verdi Protocol Analyzer* environment, *packet* and *transfer* are referred to as object types. The packets and transfers that are generated as a result of a particular simulation are referred to as objects.

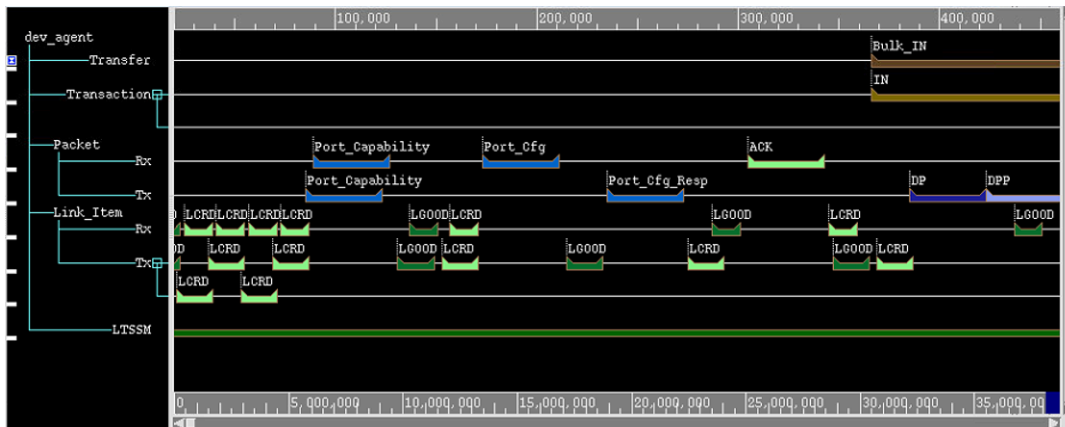
In other words, a protocol object definition is simply a way to associate a name with a collection of data that is related in some way. The motivation for defining object types in a protocol specification is to facilitate discussion of the protocol features at various levels of abstraction.

The behavior of a protocol is expressed in terms of its own specific object types. Thus, the first step to learn a new protocol is to become familiar with the structure and the behavior of those protocol-specific object types and their relationships to one another. Once an engineer becomes well-versed in the operation of a protocol, analysis or debugging of

protocol-specific behavior becomes most productive when that behavior is represented in terms of these protocol-specific objects.

In order to capitalize on such productivity, the *Verdi Protocol Analyzer* enables protocol behavior to be explored in terms of protocol-specific objects and the relationships between those objects.

Figure 5 Sample USB 3.0 Data Showing Multiple Objects of Various Object Types



Stacked Protocols or Abstraction Layers

The complexity of protocols varies greatly; for example, consider the vast difference in complexity between a basic *UART* and a *USB 3.0* host (or device). To fully explore the capabilities of the *Verdi Protocol Analyzer*, this section examines a complex protocol; specifically, the SuperSpeed aspects of the *USB 3.0* protocol.

The *USB 3.0* protocol is a *stacked* protocol, meaning that the behavior of the protocol is defined across multiple layers. These layers are (from less abstract to more abstract) the physical layer, the link layer, and the protocol layer. Complex protocols are almost always stacked. In contrast, a simple protocol is likely to be defined by only a single layer.

A protocol specification defines the activity that occurs in each layer, the data that are associated with the activity of that layer, and the manner in which the data is communicated between the layers.

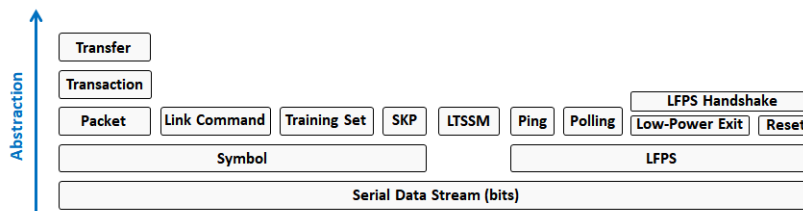
For protocols with multiple layers, perhaps the most fundamental relationship between protocol objects is the parent or child relationship. The relationship between the parent object and the child objects is one of composition. The data model of the *Verdi Protocol Analyzer* supports the specification of composite relationships, and thus understands the complete relationship tree for any particular protocol object.

A protocol specification defines the object types associated with each level of abstraction. The least abstract object types (that is, the objects types associated with the lowest level

of abstraction) are likely to correspond closely to the signal patterns observed directly on the protocol bus interface. Conversely, the most abstract object types are likely to represent complex, high-level data objects that are far removed from the detailed bus activity.

The following figure shows the fundamental object types defined by the *USB 3.0* protocol and the relationships between those object types in terms of their level of abstraction. This type of figure is extremely useful for any protocol, as it not only highlights the protocol object types that are likely to be most useful when debugging protocol-related issues, but it also suggests how the protocol data can be organized and presented.

Figure 6 *USB 3.0 (SuperSpeed) Objects Organized by Level of Abstraction*



For example, [USB 3.0 \(SuperSpeed\) Objects Organized by Level of Abstraction](#) shows *USB 3.0* objects that are captured by the *Synopsys USB 3.0 VIP*. The objects are organized into three columns: labeled *Transfers*, *Transactions*, and *Packets*, which are ordered from left to right by decreasing level of abstraction.

This approach of organizing protocol activity according to the level of abstraction makes it easy to analyze protocol-related issues in the most efficient manner. Some issues can be most easily observed and understood at a very high level of abstraction but other issues may require analysis at lower levels of abstraction.

For most protocols, the data *objects* that are manipulated in each layer can be considered as (and thus represented by) transactions. Ideally, a VIP captures the activity related to any of these objects. As previously discussed, the data should be organized according to the abstraction layer with which the data is associated.

There may be additional activity of interest that aids the protocol debug process. For example, the *USB 3.0* protocol defines a state machine (called the *LTSSM*) that resides in the link layer. The ability to capture and display the *LTSSM* state can greatly accelerate the debug process (for example, to identify a scenario where the *LTSSM* is stuck in a particular state).

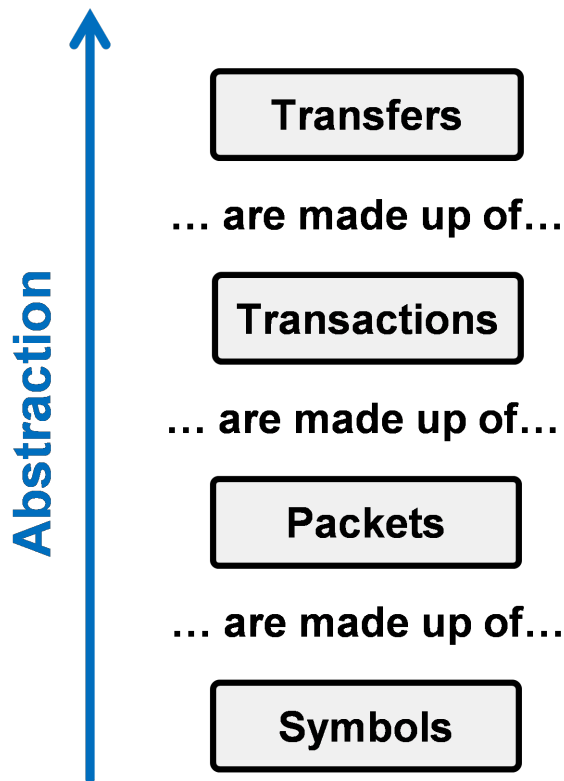
Another example of additional activity of interest that aids the debug of *USB 3.0* scenarios is the behavior of the Low-Frequency Periodic Signaling (LFPS) transmitter and receiver (that are primarily used to establish the initial communication between a host and a device and for low-power activities).

Note that both of these items are shown in the previous figure, and are located in the figure according to their relative level of abstraction. Ideally, a VIP captures this additional protocol-related data.

Parent or Child Relationships

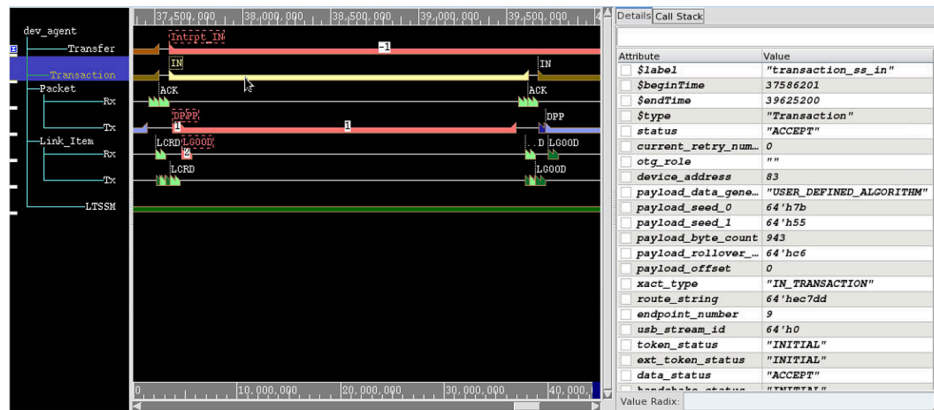
For any stacked protocol (and even for many simple protocols), there is a hierarchical relationship between objects. For example, for *USB 3.0*, a transfer is composed of one or more transactions; a transaction is composed of multiple packets; and a packet is represented by a specific sequence of symbols. As *USB 3.0* supports multiple concurrent transfers and transactions, it is extremely important to understand the relationship between these objects when debugging.

Figure 7 Hierarchical Relationships Between USB 3.0 (SuperSpeed) Objects



The following figure shows *USB 3.0* objects that are captured by the Synopsys *USB 3.0 VIP* (the objects are ordered from top to bottom by decreasing level of abstraction):

Figure 8 Hierarchical Relationships Between Objects Shown in the Verdi Protocol Analyzer



This figure shows the selection of an *IN Transaction* within the *Verdi Protocol Analyzer*. The transaction has one parent object (a *Transfer*) and three child objects (two *Packets* and one *Link Command*), which are indicated by dotted borders.

Hierarchical Object Types

An additional complexity is that the object types (such as transfers, transactions, packets, and symbols in the case of *USB 3.0*) are rarely monolithic. That is, it is common for objects of a particular type to have multiple possible representations. For example, there are different kinds of *USB 3.0* transfers, transactions, packets, and so on. When debugging a protocol, it is not only important to know the type of each object, but it is also important to be able to distinguish between the various sub-types of objects of a particular type.

[USB 3.0 \(SuperSpeed\) Objects Organized by Level of Abstraction](#) provided a comparative view of the *base* object types (such as *transfers*, *transactions*, and *packets*). These base object types are used to define the general behavior of the protocol.

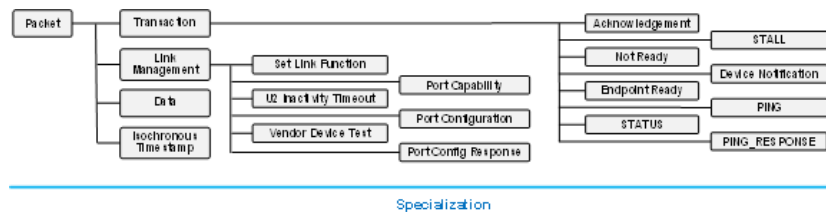
Each object type is used to discuss a common set of functionality, but for the majority of these object types, the protocol also defines more strictly-defined variations of the object type that are applicable to a specific task. In a protocol extension, each variation is referred to as an object sub-type. This is equivalent to a specialization in the software realm.

Note that in [Hierarchical Relationships Between Objects Shown in the Verdi Protocol Analyzer](#), the four child objects are shown in the lower-right portion. Even though the child objects are all *USB 3.0* transactions, each is a different kind of transaction. Also note that these transactions have different labels: *SETUP*, *OUT*, and *STATUS*. These three object types are examples of *USB 3.0* transaction object sub-types.

In the *USB 3.0* protocol, there is not only an object type hierarchy for transactions, there is also an object type hierarchy for packets. A *packet* is a *base* object type, with certain characteristics. However, the protocol later describes four different packet sub-types: *Link Management Packet (LMP)*, *Transaction Packet (TP)*, *Data Packet (DP)*, and *Isochronous Timestamp Packet (ITP)*. Two of these packet sub-types (LMPs and TPs) also have a number of sub-types.

The following figure shows all of the different USB 3.0 packet object types and how they relate to one another:

Figure 9 Hierarchical Object Sub-Type Relationships for USB 3.0 (SuperSpeed) Packets



For any particular protocol, the object type hierarchy can be multiple levels deep, depending on the complexity of the protocol.

Object Fields or Attributes

Any data associated with the operation of the protocol defines the attributes that provide detailed information about the specific protocol objects that are captured. At the most basic level, each object that is captured by the VIP must have attributes to specify the start time, the end time, and the type of object for use during debug.

In addition, protocol specifications typically define other attributes that are associated with the different object types that are manipulated by the protocol.

For example, *USB 3.0* transaction packets have a number of associated attributes, such as an endpoint number and a sequence number. Even though these attributes are ultimately represented by a number of bits in the transaction object, it is much easier to debug transaction-related issues when the values of these attributes are easily observable.

Figure 10 Example USB 3.0 Transaction Packet

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Device Address								Route String																Type				DWORD 0			
Reserved				Seq Num				NumP				HE		Rsvd		Ept Num				D		rty		Rsvd		SubType			DWORD 1		
Reserved		PP		Reserved								Stream ID/Reserved																		DWORD 2	
Link Control Word								CRC-16																DWORD 3							

The ability to view the values of object attributes is an essential aid to debug protocol-related issues. Not only are these attributes made available for analysis, each attribute is typed in order to provide robust search and filtering capabilities within the *Verdi Protocol Analyzer* debugging environment. That is, by typing an attribute as an integer, a [simulation] time, a floating-point number, or a string (for example), sophisticated searching and filtering capabilities can be provided. For example, applying comparison operators (such as '<' and '>') to an attribute whose values are represented as strings can produce completely different results than applying the same operators to an attribute (with the same values) whose values are represented as integers.

Predecessor or Successor Relationships

Hierarchical relationships are previously discussed. These are composite relationships, as they describe how one object is composed of other objects. This document uses the familiar term for a composite relationship: a parent/child relationship.

But some protocols can have a different kind of relationship between objects that is causal in nature. In other words, one object is the reason for (or cause of) the creation of one or more other objects. The more familiar term for a composite relationship is a predecessor/successor relationship.

To explain the value of being able to visualize such relationships, a different protocol is considered; namely *ACE (AXI Coherence Extensions)* of the *AMBA 4* protocol from *ARM*. This protocol enables system-level cache coherency across clusters of multi-core processors. That is, the system can utilize multiple components that each has a local memory cache. The use of a local cache can greatly increase the performance of an individual component. However, in order to maintain high performance of the overall system, it is necessary to ensure the caches of the various local components are synchronized. It is also necessary to maximize the use of cached data across the system, so as to minimize access to the main memory, which is comparatively expensive in terms of performance.

In the *AMBA 4* protocol, these coherency checks and optimizations are performed by a system interconnect (to which the various components are connected) along with special transactions (called snoop transactions), that enable the system interconnect to query other components about their local cached data.

For example, a simple *AMBA 4* might contain two components, each of which has a cache. These two components are connected to the system interconnect, along with the main memory. If the first component needs to read the value of a particular *variable*, it issues a read transaction and transmits that transaction to the system interconnect. The system interconnect, in turn, generates a snoop transaction and transmits that transaction to the second component to essentially "ask" that component if it already has a valid value for the variable of interest. The second component generates a snoop response transaction which either includes the requested data or indicates that it does not have that

While many details of the ACE protocol are omitted from the explanation above, the purpose of the example is to highlight the relationship between the two types of transactions that were introduced in that explanation: read transactions and snoop transactions.

In the scenario that was described, the snoop transaction is generated as a result of the read transaction. That is, the read transaction caused the creation of the snoop transaction. The *AMBA 4* protocol supports multiple, concurrent read transactions, and write transactions. Due to this complexity, it is useful to understand the relationship between coherent non-snoop transactions and the corresponding snoop transactions that are generated by the system interconnect.

Verdi Protocol Analyzer must be aware of the object type hierarchy in order to be able to display the protocol objects in the most meaningful manner for the debugging process. For example, all packets should be grouped together, independent of their sub-type.

However, in the following figure, note that not all packet objects are displayed in a single row; there are two rows, labeled *Rx* and *Tx*. In *Verdi Protocol Analyzer*, these *sub-rows* are referred to as channels, and provide a mechanism for separating objects of the same base object type into separate groups (which are then displayed in separate rows). In this case, the *Rx* channel displays packets that are received by the *USB VIP* and the *Tx* channel displays packets that the *USB VIP* has transmitted.

[illegible]

Recording Memory Operations

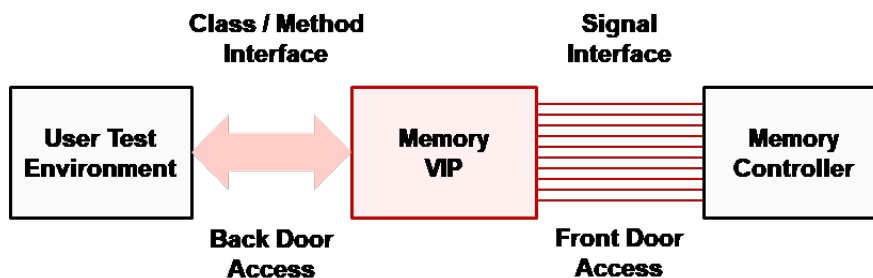
Low-Level Memory Data

There are two types of low-level memory activity that can be written to the FSDB and visualized in the *Memory Protocol Analyzer*:

- *Front door memory actions* that refer to memory activity initiated by changes to the bus (that is, as a result of *pin wiggles*)
- *Back door memory actions* that refer to memory activity initiated by the test environment.

The difference between front door and back door memory access is illustrated in the following figure:

Figure 12 Front Door and Back Door Memory Access



Memories are passive components. In other words, memories do not generate activity on their own. Memories respond to requests that are made using the memory bus.

There are two kinds of front door memory operations: *memory read* operations and *memory write* operations. Memory write operations can be further sub-divided into *normal* writes and *masked* writes. A *masked write* is the equivalent of a read-modify-write operation. Since this is now a relatively common feature of contemporary memories, the *VC Apps Protocol Analyzer APIs* support these two types of write operations and can distinguish between them. Thus, there are different memory actions for these two types of write operations. Thus, there are three *front door* memory actions: *read*, *write*, and *write_masked*.

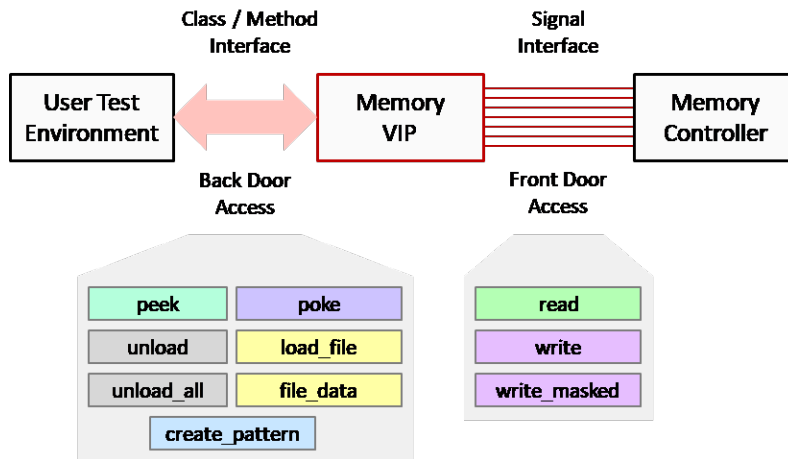
However, a robust memory VIP should provide a verification engineer with other ways to access the memory without having to access the memory bus. For example, a memory VIP should provide a mechanism for retrieving a value from memory for use in a scoreboard or injecting an incorrect data value into a specific memory location. It should also be possible for the test environment to quickly initialize a memory with a pattern or with data from a file.

The term *memory action* is used in this document to refer to the specific memory operation that is recorded in the FSDB. The following table shows the memory actions that are supported by the *VC Apps Protocol Analyzer APIs*:

Action Name	Action Type	Description
read	Front door	Indicates a single memory value was read
write	Front door	Indicates a single memory value was written
write_masked	Front door	Indicates a single memory value was written using a mask
peek	Back door	Indicates a single memory value was read
poke	Back door	Indicates a single memory value was written
unload	Back door	Unloads (that is, re-initializes) a specified address range
unload_all	Back door	Unloads (that is, re-initializes) the entire memory
load_file	Back door	Indicates memory values were loaded from a file
file_data	Back door	Indicates a specific address and value loaded from a file
create_pattern	Back door	Initializes the memory or an address range with a value or pattern

The following figure shows where front door and back door memory actions are performed:

Figure 13 Relationship Between Memory Actions and Front Door and Back Door Access



Initializing a Memory Using a Pattern

A memory (or a portion of a memory) can be initialized from the test environment (that is, with a *back door* memory action) using a pattern.

The following table describes the types of initialization patterns that are supported by the *VC Apps Protocol Analyzer APIs*:

Pattern Type	Description
CONST	Initialize the memory (or address range) with the specified constant value.
INCR	Initialize the memory (or address range) starting with the specified value and increment the value for each subsequent address.
DECR	Initialize the memory (or address range) starting with the specified value and decrement the value for each subsequent address.
WALK_LEFT	Initialize the memory (or address range) starting with the specified value and rotate the value left by one for each subsequent address.
WALK_RIGHT	Initialize the memory (or address range) starting with the specified value and rotate the value right by one for each subsequent address.

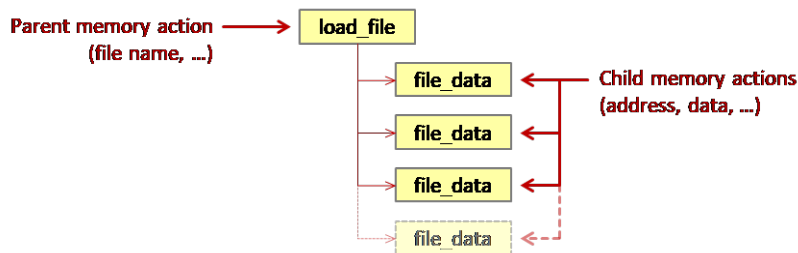
Pattern Type	Description
RAND	Initialize the memory (or address range) to random values. Because patterns are virtual, the actual random value is not assigned unless a read or peek memory action is performed on an address in the specified range.

Loading Memory Data From a File

There are two memory actions associated with loading memory data from a file. The *load_file* memory action records the name of the file from which the data is being read. This memory action must be created before recording memory actions for individual data values.

Individual address/data pairs are recorded with the *file_data* memory action. When these memory actions are created, a parent/child relationship is established with the transaction that was passed back from the method that was used to create the *load_file* memory action. These relationships are shown in the following figure:

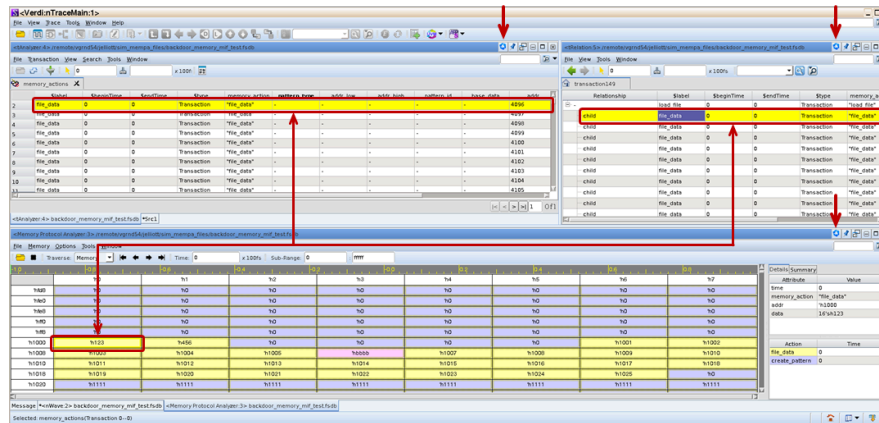
Figure 14 Relationship Between *load_file* and *file_data* Memory Actions



The data that is loaded from a file can then be cross-correlated within Verdi using *tAnalyzer*, *tRelation*, and *Memory Browser*.

The following figure shows these windows with selection synchronization enabled:

Figure 15 Selection Synchronization Between Windows for Data Loaded From a File

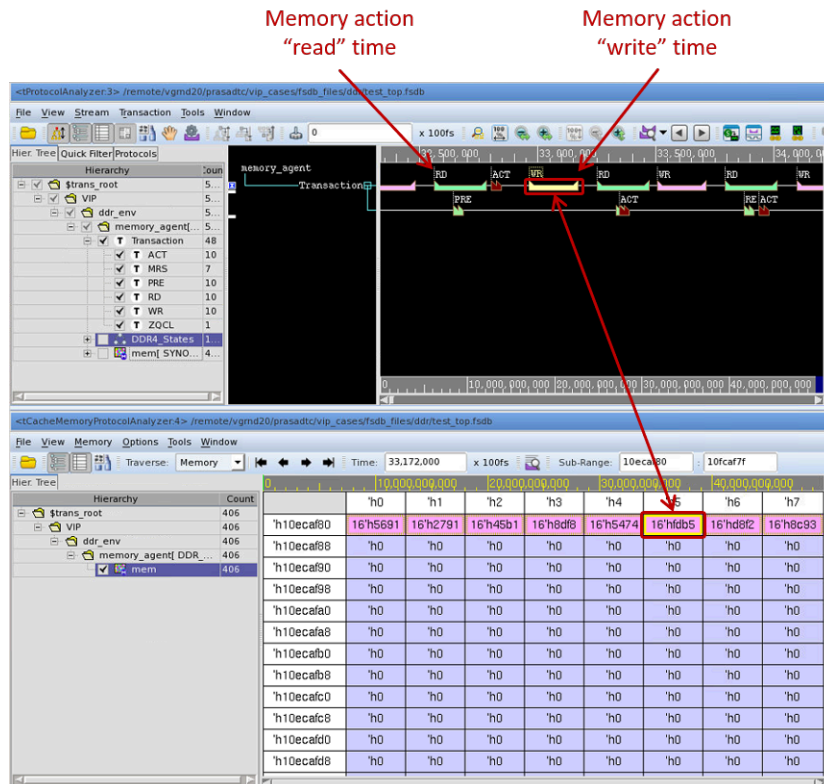


Correlating Memory Transactions With Memory Actions

For Synopsys Memory VIPs, it is possible to synchronize high-level memory protocol transactions with the subsequent front door memory actions. When selection synchronization is enabled in both *Protocol Analyzer* and *Memory Protocol Analyzer*, the selection of a protocol transaction causes a memory action to be selected, and vice-versa.

This selection synchronization is shown in the following figure:

Figure 16 Selection Synchronization Between High-Level Memory Transactions and Low-Level Memory Actions



This selection synchronization is also available for custom memory VIPs, but it requires that certain conventions be followed. As illustrated in the previous figure, the following two rules must be followed:

- For *read* memory protocol transactions, the corresponding *read* memory action must be recorded at the time the protocol transaction is started.
- For *write* memory protocol transactions, the corresponding *write* (or *write_masked*) memory action must be recorded at the time the protocol transaction is ended.

These two rules ensure that time is synchronized correctly between the two windows. In order to synchronize the memory address between the two windows, the memory protocol transaction must have an attribute called *logical_address*, whose value is the address of the memory location with which the high-level transaction is associated.

Selection synchronization only works if all of these rules are followed.

3

Integration Process

VC Apps Protocol Analyzer provides an open interface that enables integration of non-Synopsys VIPs (in-house or custom VIPs or other protocol-like components) into the *Verdi* environment so as to facilitate access to the comprehensive protocol-specific debugging capabilities provided by the Verdi platform.

In this flow, protocol-related data from these VIPs can be recorded into a common FSDB file. This data is then processed in conjunction with a protocol extension definition to enable the protocol-oriented debug and analysis capabilities of the *Verdi Protocol Analyzer*.

The *VC Apps Protocol Analyzer* integration process involves the following two steps:

1. In the first step, existing custom or in-house SystemVerilog VIP code is annotated with calls to the *VC Apps Protocol Analyzer SystemVerilog API*, which is defined in the `verdi_pa_writer` class that is delivered as part of the *Verdi* installation. This API enables the recording of protocol-related activity to an FSDB file during simulation.
2. In the second step, you can create a protocol extension definition, which describes the specific characteristics of the protocol information that was captured during simulation.

Data Recorded During Simulation

The following protocol-related information is recorded during simulation for the instances of the VIPs that are configured to capture such data:

- General VIP information
 - The location of the VIP in the design hierarchy (that is, the instance path of the VIP).
 - The name of the protocol with which the VIP is associated.
 - The version of the protocol with which the VIP is associated.
 - For memory instances, address, data width, and the memory access type.
- Protocol transactions or objects

- The common attributes of the transaction or object (start time, end time, object type, and channel name).
- Multiple, typed object attributes (with optional expected values).
- Optional composite (that is, parent or child) relationships with other objects.
- Optional causal (that is, predecessor or successor) relationships with other objects.
- Memory data
 - Front door and back door memory actions.

Protocol Characteristics

The following general protocol information is captured as part of the protocol extension definition. Note that this information is captured once for a particular protocol² as part of the integration process and is used by the *Verdi Protocol Analyzer* when processing data from any instance of that protocol.

- General protocol information
 - The name and the version of the protocol.
 - The visual attributes of the protocol, such as, display text, a brief description, and an icon.
- Optional information about methodologies supported by the VIP
 - The name of the methodology (UVM, OVM, UVM, HDL).
 - The visual attributes of the methodology, such as, display text and a brief description.
 - An optional base path or URL that indicates the location of methodology-specific help.
- Optional general protocol and/or VIP help resources
 - The name of the help resource.
 - The visual attributes of the help resource, such as, display text and a brief description.

2. *More specifically, this information described in a protocol extension definition is captured for a particular version of a protocol. Protocol definitions can change over time. Also, the information that is captured by a VIP can change over time. These possibilities are supported by the Verdi Protocol Analyzer through versioning of the protocol extension definitions.*

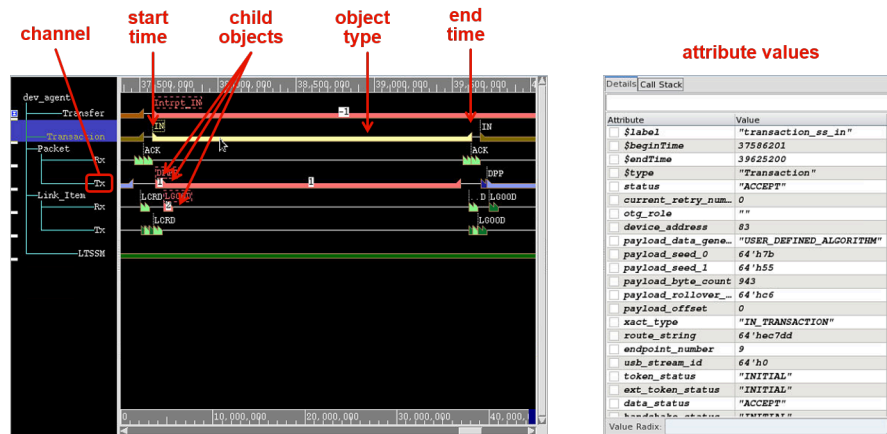
- The path or URL that indicates the location of the help resource.
- The methodology name with which the help resource is associated, if appropriate.
- Protocol object type definitions
 - The name of the object type.
 - The visual attributes of the object type, such as, display text, a brief description, an icon, and drawing attributes (background color, border color, font color).
 - The relative display order of the object type.
 - An optional SystemVerilog class name used by the VIP to represent the object type.
 - Optional object type help resources (object type help resources have the same characteristics as previously listed protocol or VIP help resources).
 - Optional protocol object type attribute definitions (see next item).
 - An optional list of protocol object sub-types (that is, names of other object types).
- Protocol object type attribute definitions
 - The name of the object type attribute (also used as the display text).
 - The data type of the object type attribute (in SystemVerilog format).
 - A brief description of the object type attribute.
 - Optional object type attribute help resources (object type attribute help resources have the same characteristics as previously listed protocol or VIP help resources).
- Signal interface groups
 - The name of the signal group.
 - The name of the signal interface with which the signal group is associated.
 - The visual attributes of the signal group, such as, the display text and a brief description.
 - The list of signals in the signal group and option radix information for each signal.

Protocol Analysis and Object Type Identification

When considering the integration of a in-house or custom VIP into the Verdi Protocol Analyzer, it is important to consider the following question: “What data can be displayed in the Verdi Protocol Analyzer?” The answer to this question is: *Almost anything*.

The reason that this answer is both very simple and yet very general is that the base Protocol Analyzer technology can be viewed as being essentially protocol-agnostic. This idea can be most easily understood by considering the basic characteristics of the object model used by the *Protocol Analyzer*.

Figure 17 Basic Characteristics of the Object Model Used by the Verdi Protocol Analyzer



The display and the analysis capabilities of the *Verdi Protocol Analyzer* can be applied to any of the *objects* that are recorded by the VIP during a simulation. While this object model shows how *objects* can easily represent protocol-related transactions, a VIP often has knowledge of other activity that can expedite the debugging and analysis of protocol behavior.

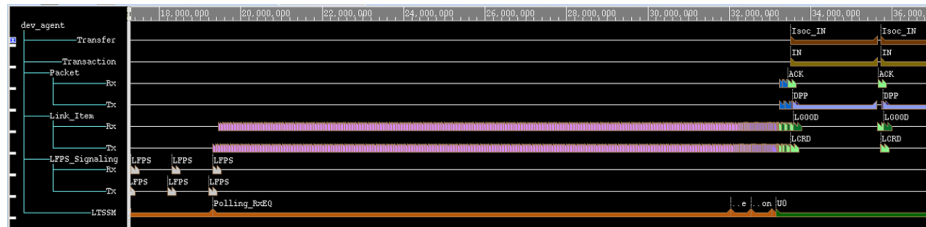
In other words, any activity of interest with respect to the operation of the VIP should be taken into consideration as a potential *object* to be recorded by the VIP during simulation. Some examples of such activity are as follows:

- Transitions of internal state machines.
- State changes for other internal structures (such as, reaching the high/low watermarks of a FIFO).
- Reconfigurations that modify the behavior of the VIP.
- Other internal VIP activity that can be represented by the object model (such as, can be viewed as having a start and end time, even if that activity is not represented by the VIP as a transaction)

[Basic Characteristics of the Object Model Used by the Verdi Protocol Analyzer](#) shows objects captured by the *Synopsys USB 3.0 VIP*. Some of these objects are transactions, such as *Transfers*, *Transactions*, *Packets*, and *Link Items*. However, note the two top-level rows at the bottom: *LFPS Signaling* and *LTSSM*. This additional information can be very useful when debugging certain types of *USB 3.0* scenarios, such as *link training* and *role swapping* (that is, OTG functionality). Note that the *LFPS Signaling* column is sub-

divided into two different channels to show difference between transmission and reception of LFPS Signaling in a clear manner.

Figure 18 Types of Objects Recorded by the Synopsys USB 3.0 VIP



Thus, the first step of the integration process is to evaluate the protocol (and the VIP) to identify the types of objects associated with the protocol (and the VIP). The goal of this step is to identify the protocol-related data that most enhances protocol-oriented debug and analysis productivity.

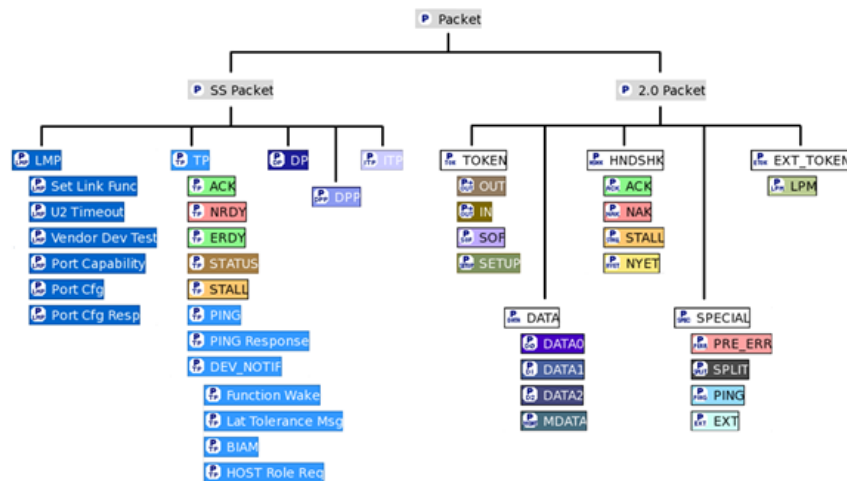
During this process, note the following points:

- Consider both *big picture* protocol-centric debugging and the detailed protocol analysis.
- List any *activity of interest* that aids protocol debug.
- Start by listing transactions (on the VIP or *layer* boundaries).
- Consider other internal activity and processing performed by the VIP.
- Consider the relationships between objects (both parent/child and predecessor/successor).

Once the top-level object types are identified, the next step in the process is to sub-divide each of the top-level object types into object type hierarchies.

The following figure shows the complete object type hierarchy for the *packet* object type (that is, for both *SuperSpeed* and *USB 2.0*), as defined in the protocol extension definition of the *Synopsys USB 3.0 VIP*:

Figure 19 Complete packet Object Type Hierarchy for Synopsys USB 3.0 VIP



An object type hierarchy is analogous to a class hierarchy. Child object types (or sub-types) are variants of their parent type. For example, a *Port Cfg Resp* packet is also an *LMP* packet, an *SS Packet*, and a *Packet*.

An object type hierarchy can greatly simplify protocol debug. Note that the different packet sub-types have different display text. This text is drawn within the rectangle that represents each packet in the timeline display within the *Protocol Analyzer*. Different object types can also have different colors; the different colors can be used to indicate different behavior. For example, for both *SuperSpeed* and *USB 2.0* packets, *bad* responses (*NRDY* and *NAK*, respectively) are drawn in a light red color, whereas *good* responses (*ACK* and *ERDY*) are drawn in a light green color. Using this differentiation, the general flow of traffic can be quickly ascertained with a quick look at the timeline display.

A robust object type hierarchy also enhances searching, filtering, and general analysis capabilities that are available in the *Protocol Analyzer*. For example, the timeline display can be easily configured to only display certain types of packets. The object type hierarchy and the number of objects of each type are also shown in the *Summary* tab.

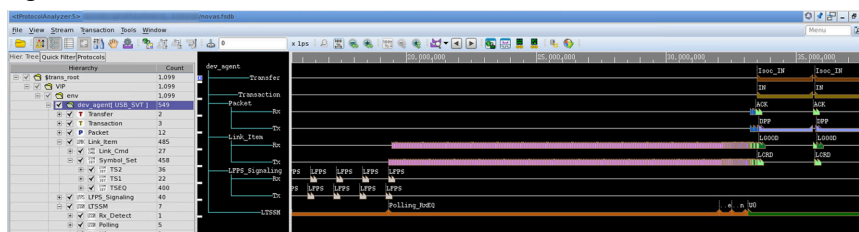
Figure 20 Summary Tab Showing Object Type Hierarchy and Number of Each Object Type

Hier	Count
Strans_root	5,092
VIP	687
dev_subenv(USB_SVT)	1,405
Transfer	12
SS_Transfer	12
Control	4
Intrpt_OUT	2
Intrpt_IN	5
Isoc_OUT	1
Transaction	20
SS_Transaction	20
IN	7
OUT	4
SETUP	5
STATUS	4
Packet	91
SS_Packet	91
Port_Capability	10
Port_Cfg	5
Port_Cfg_Resp	5
DP	17
DPP	17
ACK	26
STATUS	4
HOST_Role_Req	7
Link_Item	4,224
Link_Cmd	209
LGOOD	83
Rx	42
Tx	41
LCRD	112
Rx	56
Tx	56
LUP	14
Tx	14
Symbol_Set	4,015
TS2	140
Rx	70
Tx	70
TS1	3,812
Rx	42
Tx	3,770
TSEQ	63
Rx	31
Tx	32

Even if all of the different kinds of packets are represented in the VIP by a single SystemVerilog class, the different kinds of packets can still be recorded as different object types when they are written to the FSDB. For example, if a single class is used to represent every kind of packet, but that class includes a field to distinguish the various kinds of packets (perhaps defined using an enumerated type), the code that is added to the VIP to record packets can check the value of that field and assign the appropriate corresponding object type when creating the packet object.

As previously stated, the VIP can be instrumented to record other *non-transaction* protocol activity that is of use during the debug process. For example, it is very useful to be able to observe the state changes that occur for the state machine that is described in the link layer of the *USB 3.0* protocol (called the LTSSM), particularly during certain operations, such as SuperSpeed role swapping (where a *USB SuperSpeed* host and device switch roles).

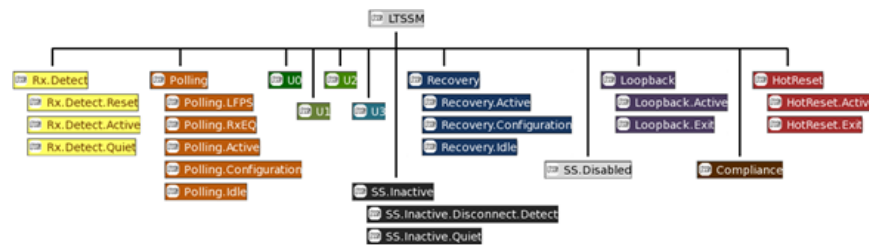
Figure 21 *USB 3.0 SuperSpeed Role Swapping Zoomed In to Show Post-Role Swap Link Training*



The bottom column of the timeline display shows the changes to the state of the LTSSM over time. To utilize this approach, the individual states of a state machine can each be represented by a different object type. For the *Synopsys USB 3.0 VIP*, the LTSSM states are represented by a multi-level hierarchy of objects, which is partially shown in the *Summary* tab on the right side of the [Summary Tab Showing Object Type Hierarchy and Number of Each Object Type](#).

The entire object type hierarchy for LTSSM is shown in the following figure:

Figure 22 Object Type Hierarchy for Synopsys USB 3.0 VIP LTSSM



4

Directory Structure and Files

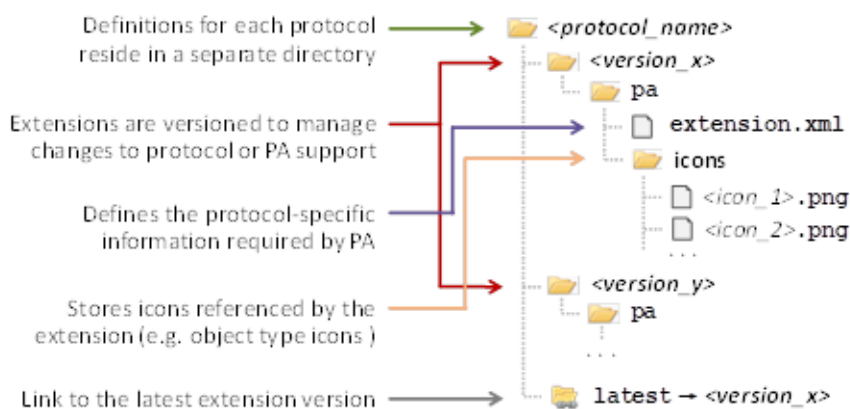
For each protocol that is integrated with the *Verdi Protocol Analyzer*, there is information about the protocol that only needs to be captured once. This information is stored in a collection of files, referred to as the protocol extension definition. The overall directory structure and files that comprise a protocol extension definition are described in this chapter.

The purpose of a protocol extension definition is to provide the *Verdi Protocol Analyzer* with protocol-specific information that enables the tool to organize and display the protocol-related data that was generated during a simulation in a manner that is most conducive to protocol-oriented debugging.

The extension definition for each protocol is contained in a directory structure. The name of the top-most directory is the internal name of the protocol. This name cannot contain any spaces or non-alphanumeric characters, with the exception of underscore ('_') characters. For simplicity, it is recommended to use all lower-case letters. To avoid naming collisions, consider adding a prefix or suffix that clearly differentiates the protocol name (such as, `snps_xyzbus` or `xyzbus_snps`).

When a VIP (or other SystemVerilog code) is annotated using the `verdi_pa_writer` class, the protocol name that is specified in the class constructor must match the name that is specified for the top-level of the protocol extension definition directory.

Figure 23 Directory Structure for a Protocol Extension Definition



The next level in the directory structure of the protocol extension definition is a sub-directory that represents the *version* of the protocol extension definition.

Every protocol extension definition is versioned that enables changes or enhancements to be made to a protocol extension definition without the risk of breaking the existing flows of current users. For example, a VIP may be enhanced to capture additional protocol-related data during simulation. This new data may come from new features that are added to the protocol over time; or the new data may come from end-user requests that additional information be captured during simulation. Either of these scenarios would certainly require modifications to the protocol extension definition.

The version string cannot contain any spaces or non-alphanumeric characters, with the exception of underscore ('_') and dot('.') characters. For simplicity, it is recommended to use all lower-case letters. While any version string is permitted, a simple versioning scheme is recommended (such as, *1.0*, *1.1*, *2.0_beta*, *2.0*, and so on).

For each version of the VIP that has been annotated using the `verdi_pa_writer` class, there should be a corresponding version of a protocol extension definition for that VIP (that is, a sub-directory whose name is the same as the version string passed to the `verdi_pa_writer` constructor in the VIP SystemVerilog code). Note that when a VIP (or other SystemVerilog code) is annotated using the `verdi_pa_writer` class, both the protocol name and the protocol version are specified in the class constructor.

Figure 24 Sample Instantiation of the `verdi_pa_writer` Class

```
...
begin
    verdi_pa_writer pa_writer;
    ...
    pa_writer = new( get_full_name(), "xyzbus_snps", "1.1" );
    ...
end
```

As shown in [Directory Structure for a Protocol Extension Definition](#), there is a symbolic link called “latest” at the same level in the directory structure as the extension definition versions. The version of the protocol extension definition that this link points to indicates the default version of the extension definition.

This link is only utilized if there is no directory that corresponds to the information captured during simulation (that is, there is no directory that corresponds the version reported by the VIP). If the versions of both the annotated VIPs and the protocol extension definitions are managed appropriately, this situation should not occur. Also, note that under these conditions, errors or warnings may be generated when the data is read into the *Verdi Protocol Analyzer*.

Within the version sub-directory is another sub-directory that must be called *pa*. This additional layer is included in the directory hierarchy to separate information related to the *Verdi Protocol Analyzer* from any other protocol-related data. While this is completely optional, this allows the protocol extension definition directory structure to be used as a

common location to store other protocol-related files, such as, documentation, code, or any other data, if so desired.

The *pa* directory contains the following two items:

- A file - *extension.xml*. The *extension.xml* file is an XML file (as implied by its extension) and defines all of the protocol-related information required by the *Verdi Protocol Analyzer*.
- A sub-directory - *icons*. The *icons* directory contains icons that provide visual representations for protocol-specific object types that are defined for the protocol. These icons are used by the *Verdi Protocol Analyzer* throughout the GUI to provide visual cues to aid protocol debug.

The following figure shows the various object type icons for the *Synopsys USB 3.0 VIP*:

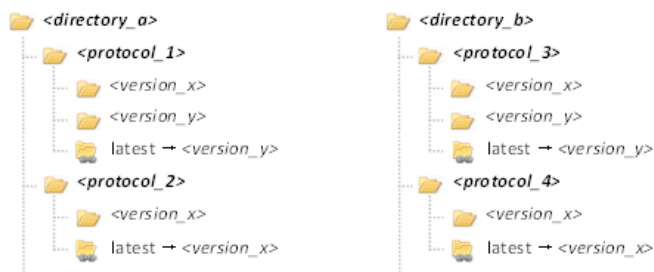
Figure 25 Object Type Icons for Synopsys USB 3.0 VIP

Hier	Count
\$trans_root	5,092
VIP	687
dev_subenv[USB_SVT]	4,405
host_subenv[USB_SVT]	12
Transfer	12
SS_Transfer	4
Control	2
Intrpt_OUT	5
Intrpt_IN	1
Isoc_OUT	20
Transaction	20
SS_Transaction	7
IN	4
OUT	5
SETUP	4
STATUS	91
Packet	91
SS_Packet	10
Port_Capability	5
Rx	5
Tx	5
Port_Cfg	2
Rx	3
Tx	5
Port_Cfg_Resp	3
Rx	2
Tx	17
DP	9
Rx	8
Tx	17
DPP	9
Rx	8
Tx	26
ACK	10
Rx	16
Tx	4
STATUS	3
Rx	1
Tx	7
HOST_Role_Req	4
Rx	3
Tx	4,224
LINK Link_Item	28
LFPS LFPS_Signaling	30
LTSSM	

Location of Protocol Extension Definitions

Protocol extension definitions can reside in one or more directories. These directories are specified by setting the value of the `PA_CUSTOM_VIP_PATH` environment variable. Multiple directory paths can be specified using the colon character (:) as a delimiter.

Figure 26 Multiple Protocol Extension Definitions in Multiple Locations



[Multiple Protocol Extension Definitions in Multiple Locations](#) shows four protocol extension definitions, represented as <protocol_1>, <protocol_2>, <protocol_3>, and <protocol_4>.

The protocol extension definitions for <protocol_1> and <protocol_2> are located in a directory represented by <directory_a>. The protocol extension definitions for <protocol_3> and <protocol_4> are located in a directory represented by <directory_b>.

For this scenario, the PA_CUSTOM_VIP_PATH environment variable must be set as follows:

```
%setenv PA_CUSTOM_VIP_PATH <directory_a>:<directory_b>
```

The value of the PA_CUSTOM_VIP_PATH environment variable must be set prior to invoking the *Verdi Protocol Analyzer*.

5

Protocol Extension XML File

The protocol extension file (*extension.xml*), which resides in the *pa* sub-directory of a protocol extension definition is an XML file (as implied by its extension) and defines all of the protocol-related information required by the *Verdi Protocol Analyzer*.

Since this file resides in the *pa* sub-directory, which itself resides in the version sub-directory of a protocol directory, the extension file defines the characteristics of a specific version of a protocol.

The extension file utilizes a standard XML (eXtensible Markup Language) format. XML is a “human-readable” (that is, ASCII) format that appears very similar to HTML.

This chapter does not explain all the details of what can be specified using XML, as the extension file only utilizes the simplest XML constructs. However, this chapter attempts to explain enough of the “basics” of XML, so that an extension file can be constructed without the need for external research.

This chapter contains the following sections:

- [XML Basics](#)
- [General Protocol Information](#)
- [Supported Methodologies](#)
- [Protocol or Methodology Help Resources](#)
- [Object Types or Object Type Hierarchies](#)
- [Object Type Fields or Attributes](#)
- [Signal Interface Groups](#)

XML Basics

As previously mentioned, the *extension.xml* file utilizes the XML format, which is ASCII, and thus can be created and modified with any text editor. The extension file can be created “brute force”, by just typing in the constructs that are described in this document. However, this document has an appendix that describes tools and methods that can be

used to simplify the construction and validation of protocol extension files; the review of this appendix is highly encouraged.

In fact, many of the details that are described in this section are managed by tools that are freely available on the Internet. The basics of the XML format are described in this document nonetheless: for completeness, for help with understanding issues or problems that are encountered, and for those who may simply choose to utilize their favorite text editor (that may not provide specific support for XML).

Every XML document should begin with an XML declaration, which describes the XML format:

```
<?xml version="1.0" encoding="UTF-8"?>
```

The *data* described by the XML file is specified using tags, elements, and attributes.

A tag is a string that begins with the *less than* (<) character and ends with the *greater than* (>) character. There are three kinds of tags: start tags, end tags, and empty element tags. Start tags contain strings that can contain only alphanumeric characters and underscores, such as <protocol> (the exact restrictions are more complex than this, but all of the tags that appear in an extension file follow this rule). End tags are similar, but also include a forward slash (/) character after the starting < character, such as </protocol>. Empty element tags are also similar to start tags, but include a space and a forward slash before the ending > character, such as <help_resource />.

An element is an XML construct that either has matching start and end tags or is an empty element tag (such as <protocol> </protocol> and <help_resource />). Elements that are defined with matching start and end tags can include content between those tags. This enables XML to describe hierarchical data relationships. Elements that are included within other elements are referred to as child elements.

An attribute is a name-value pair that can be defined within a start tag or an empty element tag. Attribute names can only contain alphanumeric characters and underscores. The attribute value is specified after the attribute name using an *equals* (=) character, followed by a string that must be enclosed within double-quotes ("").

The following *methodology* empty element tag defines four attributes: *name*, *display_text*, *description*, and *base_help_url*:

```
<methodology name="uvm"  
             display_text="UVM"  
             description="Universal Verification Methodology"  
             base_help_url="$VIP_HOME/sfp/1.1/doc/uvm/html/" />
```

An XML file can also include comments. Comments can be included in XML files at the same locations where elements are allowed. A comment starts with the "<!-- " (note

the space after the two dashes) string and ends with the “-->” string. Comments cannot contain “--” (two dashes together), which means that comments cannot be nested. For example, the previous example might be preface with the following comment line:

```
<!-- Supported Methodologies -->
```

Since XML uses the characters ‘<’, ‘>’, ‘”’, and ‘&’ as formatting delimiters, these characters cannot appear directly in attribute value strings. Instead, these characters must be “escaped”. These four characters and their corresponding replacement strings are shown in the following table:

Character	Replacement String
<	<
>	>
"	"
&	&

For example, the following XML shows how the double-quotes in the value of the *text* attribute have been replaced:

```
<description text="&quot;lmp_subpkt_type&quot; is common to all LMP  
subtypes" />
```

While this section provides a very simplified introduction to XML, it provides the basic information that is needed to construct a protocol extension file.

General Protocol Information

The *protocol* element is the outermost element of the protocol extension XML structure. The values of the attributes associated with this element are used to specify general information about the protocol.

The attributes associated with the *protocol* element are shown in the following table (all three of these attributes are required³):

Attribute	Description
name	The “internal” name of the protocol, which should be the same as the name of the top-most directory of the protocol extension definition
display_text	The text to display when information pertaining to the protocol is shown in the UI
description	A brief (that is, usually one-line) description of the protocol

The following XML code shows the outer-most structure of a protocol extension XML file:

```
<?xml version="1.0" encoding="UTF-8"?>
<plugin>
  <extension point="com.synopsys.sg.vip.pa.protocol">
    <!--
      Verdi Protocol Analyzer Extension Definition
      for Synopsys SuperFast Protocol (version 1.1)
    -->
    <protocol name="snps_sfp"
      display_text="Synopsys SFP"
      description="Synopsys SuperFast Protocol VIP">
      . . .
    </protocol>
  </extension>
</plugin>
```

All other information about the protocol is represented as child elements of the *protocol* element. Each of these elements is described in the sections that follow.

Supported Methodologies

The *methodology* element describes the attributes of each methodology that is supported by the VIP. For example, most Synopsys VIPs support multiple methodologies, such as, UVM, OVM, VMM, and HDL (the latter enables the VIP to be inserted into a “pure Verilog” verification environment).

3. *The names of optional attributes are drawn in the attribute tables in italics. The section that contains the table also discusses whether an attribute is required or optional (and, if optional, the condition under which it should be defined). The italic text simply provides a simple visual queue when referencing this document at a later time.*

The attributes associated with the *methodology* element are shown in the following table:

Attribute	Description
name	The "logical" name of the methodology that the VIP supports. Currently, the value of this attribute is restricted to "uvm", "ovm", "vmm", or "hdl".
display_text	The text to display when information pertaining to the methodology is shown within the UI.
description	A brief (that is, usually one-line) description of the methodology (this can provide clarity to the user within the UI, if needed)
base_help_url	The location of the base folder (or URL) for class reference documentation. (The use of this attribute is described in detail later in this document)

The *name*, *display_text*, and *description* attributes must be specified for each methodology. The *base_help_url* attribute is optional (more on this topic later).

The following XML code shows *methodology* elements for a VIP that supports two methodologies (UVM and OVM). Note that these two elements are located within the outer-most *protocol* element. The *protocol* element has been truncated and drawn in grey to highlight the methodology definitions.

```
<protocol . . . >

  <!-- Supported Methodologies -->
  <methodology name="uvm"
    display_text="UVM"
    description="Universal Verification Methodology"
    base_help_url="$VIP_HOME/sfp/1.1/doc/uvm/html/" />

    <methodology name="ovm"
      display_text="OVM"
      description="Open Verification Methodology"
      base_help_url="$VIP_HOME/sfp/1.1/doc/ovm/html/" />

    . . .
</protocol>
```

The *base_help_url* attribute is an optional attribute that can provide the following two benefits:

- Firstly, it greatly simplifies the process of defining help resources for common protocol items, such as, object types and the fields (that is, attributes) that are associated with object types.
- Secondly, for VIPs that support multiple protocols, it also simplifies access to those help resources from within the *Verdi Protocol Analyzer*.

The details of this mechanism are discussed in a later section of this document, specifically in the sections that describe how to define object types and their associated fields.

For each methodology that the VIP supports, there should be a corresponding *methodology* element in the extension definition. Even if the VIP that is being defined only supports one methodology (UVM, for example), a corresponding *methodology* element should still be added to the extension definition.

Protocol or Methodology Help Resources

During the debug process, it is important to have easy access to help resources. Help resources can be general or specific to a VIP. Further, VIP-specific help may only apply to a particular methodology, such as, UVM or OVM.

The help resources that are defined in a protocol extension can reference any file that can be viewed in a browser or PDF viewer. Further, the help resource can reference a "local" file or an "external" URL. Some examples of help resources are VIP manuals, VIP class reference material, Application notes, methodology guides, and user group sites.

The *help_resource* element defines a help resource using the attributes shown in the following table:

Attribute	Description
methodology	The methodology with which the help resource is associated. This attribute is optional and should not be specified for general help resources. The value of this attribute is restricted to "uvm", "ovm", "vmm" or "hdl".
name	The internal name to use when referring to the help resource. This name cannot contain any spaces or non-alphanumeric characters, with the exception of underscore ('_') characters. For simplicity, the use of all lower-case letters is recommended.
display_text	The text to display when information pertaining to the help resource is shown within the UI.

Attribute	Description
description	A brief (that is, usually one-line) description of the help resource (this can provide clarity to the user within the UI, if needed).
location	The location of the help resource. This can be a path to a file or a URL and can include environment variables.

The *name*, *display_text*, *description*, and *location* attributes must be specified for each help resource. As noted in the table, the *methodology* attribute is optional and should only be specified for methodology-specific help resources.

The following XML code shows specifications for two general protocol help resources. Note that these two elements are located within the outer-most *protocol* element. The *protocol* element has been truncated and drawn in grey to highlight the methodology definitions.

```
<protocol . . . >

  <!-- General Protocol Help Resources -->
  <help_resource name="release_notes"
    display_text="SFP VIP Release Notes"
    description="Synopsys SuperFast Protocol VIP
    Release Notes"
    location
   ="$VIP_HOME/sfp/1.1/doc/spf_release_notes.pdf" />

  <help_resource name="solvNetPlus"
    display_text="Synopsys SolvNetPlus"
    description="Access to Synopsys SolvNetPlus on-line
    help resources"
    location="https://solvnetplus.synopsys.com" />
    . . .

</protocol>
```

Note that the location of the first help resource is a local file whose location is determined by the value of an environment variable and the location of the second help resource is a URL.

The following XML code shows specifications for two methodology-specific help resources:

```
<protocol . . . >

  <!-- Methodology-Specific Help Resources -->
  <help_resource methodology="uvm"
    name="uvm_user_manual"
    display_text="SFP VIP UVM User Manual"
```

Chapter 5: Protocol Extension XML File
Protocol or Methodology Help Resources

```
description="Synopsys SuperFast Protocol VIP UVM User
Manual"
location
="$VIP_HOME/sfp/1.1/doc/spf_uvm_user_manual.pdf" />

<help_resource methodology="ovm"
name="ovm_user_manual"
display_text="SFP VIP OVM User Manual"
description="Synopsys SuperFast Protocol VIP OVM User
Manual"
location
="$VIP_HOME/sfp/1.1/doc/spf_ovm_user_manual.pdf" />
. . .
</protocol>
```

Both general protocol and methodology-specific help resources can be accessed from the *Protocols* tab in the *Verdi Protocol Analyzer*.

HTML help resources can be opened either in a tab which is internal to the *Protocol Analyzer* or in an external browser, depending on the user's preference settings.

Figure 27 Help Resources Shown in an Internal Browser

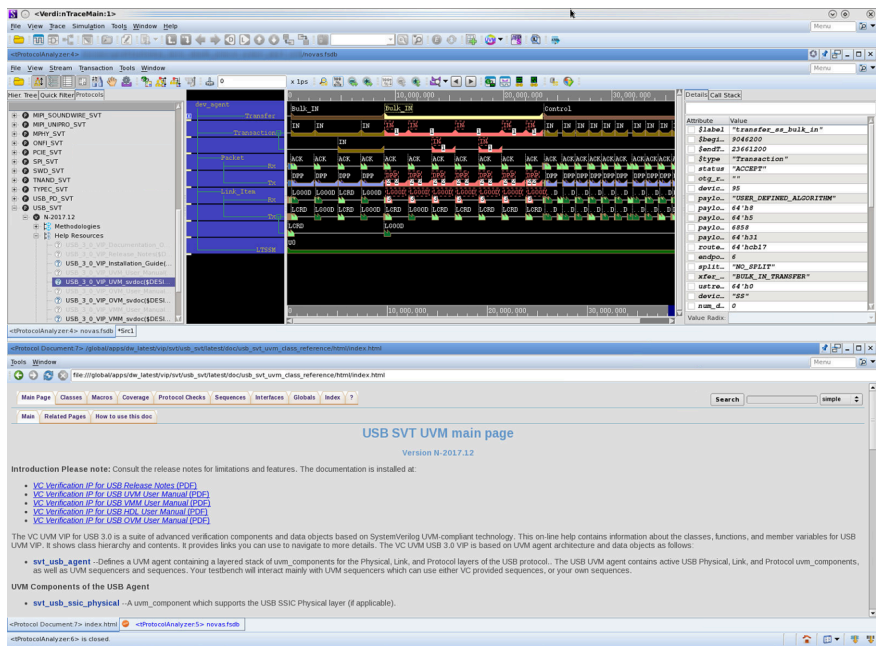
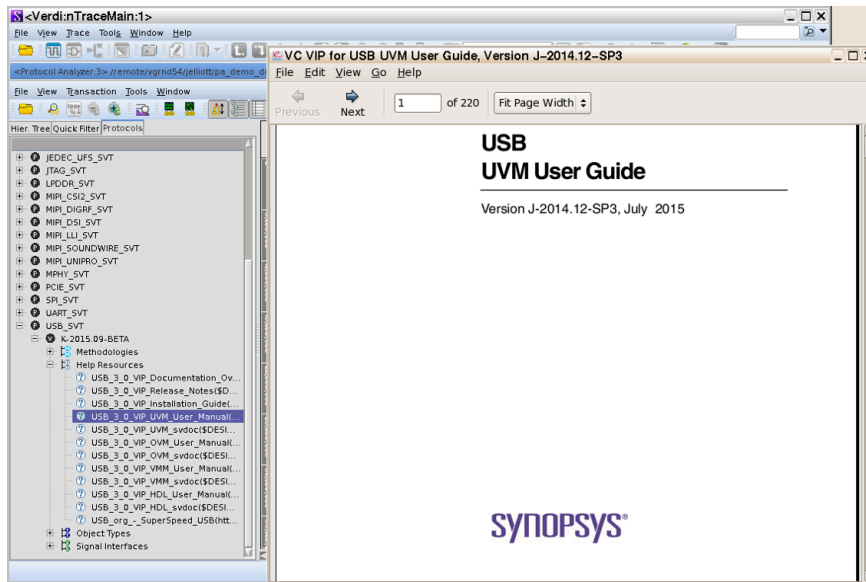


Figure 28 Help Resources Shown in an External PDF Reader



Object Types or Object Type Hierarchies

The *object_type* element defines an object type using the attributes shown in the following table:

Attribute	Description
name	The internal name to use when referring to the object type. This name cannot contain any spaces or non-alphanumeric characters, with the exception of underscore ('_') characters. For simplicity, the use of all lower-case letters is recommended.
display_text	The text to display when information pertaining to the object type is shown within the UI.
description	A brief (usually one-line) description of the object type (this can provide clarity to the user within the UI, if needed). The description can also be defined using a sub-element. This approach is shown in the example that follows.
background_color ⁴	The background color to use when drawing objects of this type in the timeline and other displays (such as, search results).
border_color ⁴	The border color to use when drawing objects of this type.
font_color ⁴	The font color to use when drawing objects of this type.

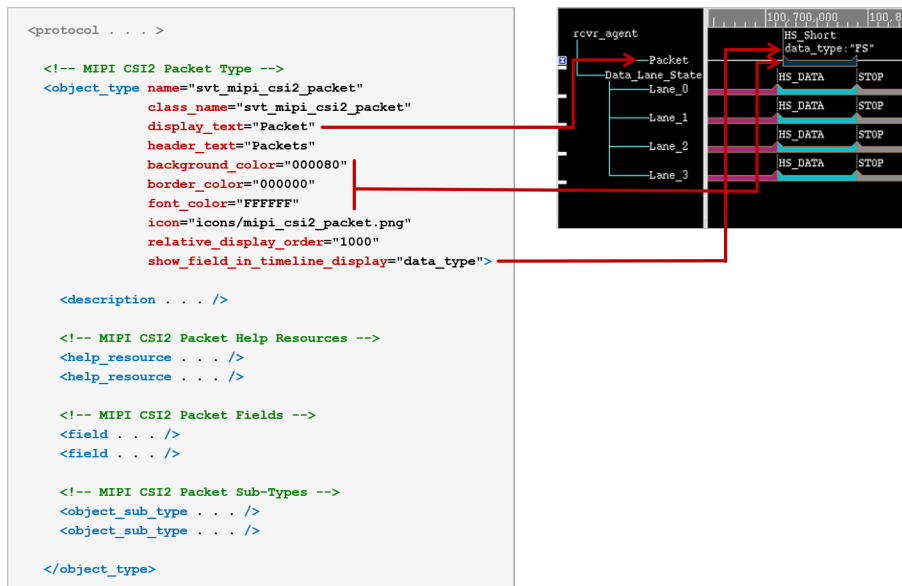
Attribute	Description
icon	The location of the icon to use to represent this object type. The icon should be a 16x16 PNG image located in the “icons” sub-directory.
relative_display_order	An integer that indicates the location of this object type relative to other object types in the same hierarchy or sub-hierarchy when drawn in the timeline display or lists.
show_field_in_timeline_display ⁵	An optional list of comma-separated attributes, whose values are drawn in the rectangles that represent objects of this type in the timeline display. This attribute is inherited for object sub-types (so it is not necessary to copy this attribute in all object sub-types).
class_name	An optional attribute, whose value is the name of the SystemVerilog class with which this object type is associated.
header_text ⁶	The text to display in the column of the timeline display. The <i>header_text</i> attribute is required for top-level object types, but has no meaning for object sub-types.

Note:

Colors are represented as a six digit hexadecimal number in the form RRGGBB, where “RR” represents red, “GG” represents green and “BB” represents blue. Red, green, and blue values can range from “00” (0) to “FF” (255). For example, “000000” represents black and “FFFFFF” represents white. Colors must be specified for top-level object types. If color attributes are not specified for an object sub-type, its colors are inherited from its parent object type.

The general structure of an object type is shown below. This is an example of a top-level object type, so the *header_text* attribute is specified. Also, in this specific example, the colors of the object sub-types are inherited from this top-level object type (which is shown later), so the objects within this object type hierarchy are drawn with the colors specified in this object type definition. This example also shows how the *show_field_in_timeline_display* attribute affects how objects are drawn in the timeline display.

Figure 29 Sample Object Type Definition Showing Attributes and General Sub-Element Structure



Note that in the previous example, the description is not declared as an attribute. For object types, the description can be defined using either an attribute, or alternately can be defined using a sub-element. If the *description* sub-element is used, the *text* attribute is used to define the description:

```
<description text="MIPI CSI2 Packet" />
```

In the previous example, other sub-elements are shown (however, with their attributes omitted). These optional sub-elements define help resources, fields, and object sub-types.

Two example help resources for the `svt_mipi_csi2_packet` object type are shown below. The other details of the object type definition have been removed or truncated for clarity. The methodology definitions for the protocol VIP are also shown in the following figure:

Figure 30 Example Object Type Help Resource that Utilizes Optional "methodology" Attribute



```

<protocol . . . >
  <!-- Supported Methodologies -->
  <methodology name="um"
    display_text="UVH"
    description="Universal Verification Methodology"
    base_help_url="$DESIGNWARE_HOME/vip/svt/mipi_svt/J-2014.12/doc/mipi_csi2_svt_um/html/" />
  <methodology name="umh"
    display_text="UHH"
    description="Verilog Methodology Manual"
    base_help_url="$DESIGNWARE_HOME/vip/svt/mipi_svt/J-2014.12/doc/mipi_csi2_svt_umh/html/" />

  <!-- MIPI CSI2 Packet Type -->
  <object_type name="svt_mipi_csi2_packet" . . . >

    <!-- MIPI CSI2 Packet Help Resources -->
    <help_resource name="um_svdoc"
      methodology="um"
      description="MIPI CSI2 UVH Packet Class Reference"
      display_text="svt_mipi_csi2_packet (class)"
      location="class_svt_mipi_csi2_packet.html" />
    <help_resource name="umh_svdoc"
      methodology="umh"
      description="MIPI CSI2 UHH Packet Class Reference"
      display_text="svt_mipi_csi2_packet (class)"
      location="class_svt_mipi_csi2_packet.html" />
  . . .
</object_type>
  
```

The image shows an XML snippet with red annotations. Red arrows point to the `base_help_url` attribute in the `<methodology>` elements and the `methodology` attribute in the `<help_resource>` elements. Red dashed arrows point from the `base_help_url` attribute to the `location` attribute in the `<help_resource>` elements, indicating that the base URL is prepended to the location.

During the debug process, if a problem appears to be related to a *packet* object, having easy access to the help resources associated with the *packet* object type would be very useful. However, the location of the help resource may be methodology-dependent. Further, the location of the help resources may include deep directory paths, whose duplication is error-prone. The `base_help_url` attribute associated with the *methodology* element can help address these issues.

The methodology that was used to generate simulation data can be set when the resulting data is viewed in the *Verdi Protocol Analyzer*. If the methodology is set, only the “pertinent” help resources are shown. The pertinent help resources include those that do not specify the optional *methodology* attribute or those that do specify this attribute and match the methodology specified for the design. Further, if the *methodology* element defines the option `base_help_url` attribute, that attribute is prepended to the “location” specified in the *help_resource* element.

For example, using the data in the previous figure, if the methodology for the design is set to *UVM* and help is requested for an object whose type is *svt_mipi_csi2_packet*, the following help page is displayed:

```
$DESIGNWARE_HOME/vip/svt/mipi_svt/J-2014.12/doc/mipi_csi2_svt_uvm/html/class_svt_mipi_csi2_packet.html
```

If no methodology is set for the design, both UVM and VMM help resources are presented for the user to select from.

In the previous example, there are separate help resource definitions for each methodology. This is only necessary if there is a need or a desire to have a different value for *description*, *display_text*, or *location* attributes for each help resource. For the previous example, if the *description* attribute was changed to *MIPI CSI2 Packet Class Reference*, only one help resource definition would be needed.

The ability to define object type hierarchies is a key feature of the *Verdi Protocol Analyzer*. Object sub-types are specified using the *object_sub_type* element that must be contained within an *object_type* element. Forward referencing is permitted (that is, an object type can refer to the name of another object type that has not yet been defined in the extension file).

The *object_type* element uses a single attribute called *name*, as shown in the following example:

Figure 31 General Structure of Object Sub-Type Definitions

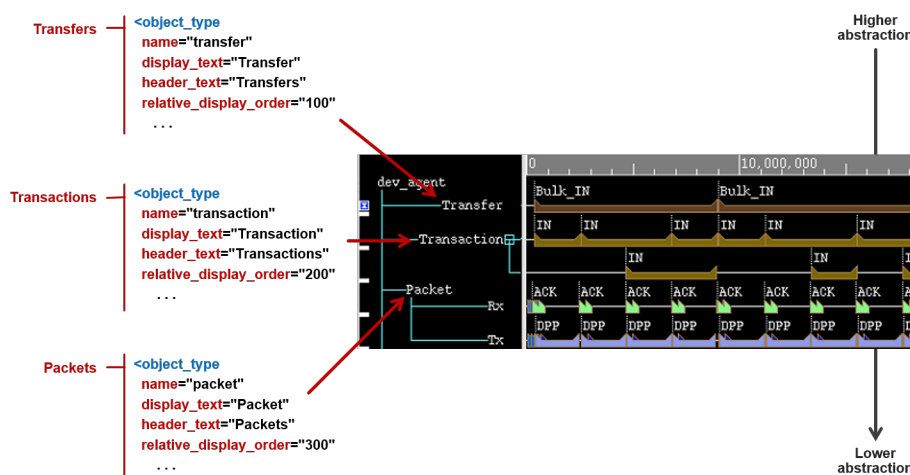


Note that the object in the first column is an object of type *svt_mipi_csi2_hs_packet*, as the text *HS* is displayed on the object. This object is drawn in the *Packets* column because its object type is a sub-type of *svt_mipi_csi2_packet* (which is a top-level object type and thus results in the creation of a column).

The ordering of an object type relative to other object types in the same hierarchy or sub-hierarchy when drawn in the timeline display or lists is determined by the value of the *relative_display_order* attribute for each object type. The order of columns in the timeline display is determined in the same manner for top-level object types.

This is shown in the following figure:

Figure 32 Ordering of Top-Level Object Types Using "relative display order" Attributes



Note that the integer values for the attributes do not have to be sequential. Leaving “space” between these values makes it easy to add new top-level object types at a later date, should the need arise.

While the top-level object types can be placed in any order, it is recommended that the most abstract top-level object types be placed in the left-most column and that other columns be placed in order of decreasing level of abstraction. This encourages debug to start with the highest level of abstract possible and then proceed to more detailed information as needed to analyze the protocol activity.

Object Type Fields or Attributes

When debugging protocol-related data, information about the objects that were generated during simulation is very important. This information includes the types of those objects, the start and end times of those objects, and the relationships between the objects. However, perhaps the most important information that is required for tracking down

the details of a protocol-related issue comes from the field values (or attribute values) associated with each object.

Field (or attribute) definitions are associated with an object type. The specification of fields is related to the object type hierarchy, in that object sub-types can specify values for fields that are defined in their parent object type declaration. In other words, the lowest-level object types in an object type hierarchy inherit the field definitions of the parent object types all the way up the object type hierarchy tree.

As previously discussed, an object type is usually associated with a particular SystemVerilog class. Similarly, if this is the case, each field for that object type is usually associated with a particular data member of that class.

The *field* element defines an object type field using the attributes shown in the following table:

Attribute	Description
name	The name to use when referring to the field. This name is also used as the display text for the field
data_type	The type declaration for the field using SystemVerilog syntax. Examples of this are bit, bit [7:0], int, and so on

While the ability to capture and display the values of object fields is an important aspect of protocol debugging, it is necessary that the attributes be typed in order to fully take advantage of the *Verdi Protocol Analyzer* protocol-oriented debugging environment. In other words, it is desirable to indicate whether the type of a field is, for example, an integer, a [simulation] time, a floating-point number, a bit-vector or a string.

The typing of attributes is important, as it enables sophisticating searching and filtering capabilities. For example, applying comparison operators (such as '<' and '>') to an attribute whose values are represented as strings can produce completely different results than applying the same operators to an attribute (with the same values) whose values are represented as integers.

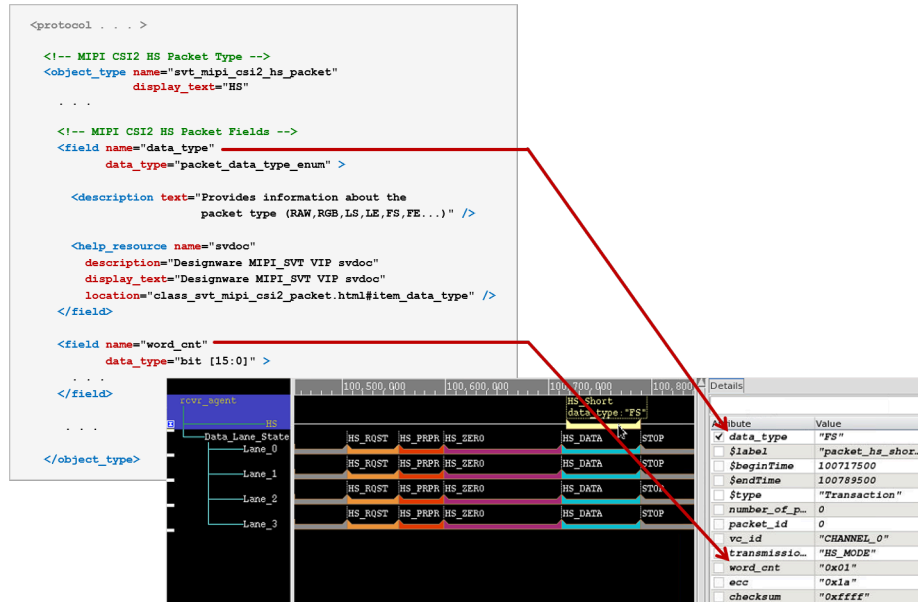
As just mentioned, SystemVerilog data types enable type-based searches that utilize comparison operators in the same manner as those comparison operators would operate in the context of a simulation.

However, the *Verdi Protocol Analyzer* does not recognize every SystemVerilog type. In the case that the specified data type for a field is not supported, the values for that field are interpreted as strings. Perhaps the most common situation in which this occurs is for field values that are defined using enumerated types. However, even when enumerated type values are interpreted as strings, it is still possible to perform sophisticated searches using those fields.

To simplify the creation of the field definition, data type modifiers (such as “rand”) can be specified in the “data_type” attribute; however, such modifiers are ignored.

Two example field definitions for the *svt_mipi_csi2_hs_packet* object type are shown below. The other details of the object type definition have been removed or truncated for clarity.

Figure 33 Example Object Type Field or Attribute Definitions



Note that object type field definitions share some structural characteristics with object type definitions with respect to the specification of the description for the field and for help resources associated with the field. Specifically, a description can be included as an attribute of the field element or can be specified as a separate element; for help resources, the same functionality previously discussed with respect to the optional *methodology* attribute and the use of *base_help_url* applies in the same manner to field help resources.

Signal Interface Groups

While the *Verdi Protocol Analyzer* provides debug and analysis capabilities at a level of abstraction that is above the waveform view, there are times when it is necessary to examine the detailed bus activity at the signal interface level, especially in the case of debugging protocol-related errors. However, a waveform view of the bus interface signals on its own is often difficult to understand, as such, a view typically contains a large amount of information with little context regarding what bus-related activity is occurring and when.

To address this challenge, the *Verdi Protocol Analyzer* can be synchronized with the Verdi waveform viewer to help correlate the protocol-level representation of the bus activity

presented in the Protocol Analyzer timeline display with the signal-level representation of the bus signal interface presented in a wave window.

To utilize this capability, the simulation of the design must have been run in a manner that captures signal change information (that is, using `$fsdbDumpvars`, or some other equivalent mechanism).

As discussed earlier in this document, signal interface groups can be associated with a protocol extension definition.

The signal interface groups are used to populate the Verdi *nWave* window with particular interface signals that are associated with the protocol. Multiple signal interface groups can be supplied with the protocol extension definition; this can be useful if the protocol supports multiple interfaces or if there are situations where it is beneficial to load only a subset of the protocol interface signals into the waveform window.

Interface signals are added to the wave window as defined by one or more signal interface groups. A protocol extension can include multiple signal interface definitions. For example, the extension definition for the AXI protocol includes a number of different signal groups to provide the end user with a choice of signal interface definitions (for example, read interface signals, write interface signals, and address interface signals).

Keep in mind that the signal interface groups are used to display protocol-related signals and thus are specific to the protocol.

The *signal_if_group* element defines a signal interface group using the attributes shown in the following table:

Attribute	Description
if_name	The name of the interface with which the signal group is associated. If the signal interface group is associated with multiple interfaces, multiple interfaces can be specified using “,” as the delimiter
group_name	The name to use when referring to the signal interface group. This name cannot contain any spaces or non-alphanumeric characters, with the exception of underscore ('_') characters. For simplicity, the use of all lower-case letters is recommended
display_text	The text to display when information pertaining to the signal interface group is shown within the UI
description	A brief (usually one-line) description of the signal interface group (this can provide clarity to the user within the UI, if needed)
extended_interface_path	If VIP clients are unable to capture the complete path for the signal interface group, this option path can be specified. If specified, the value of this attribute is appended to the interface paths that were captured during simulation (described later) and then attempt to find signals from the appended path.

Attribute	Description
signals	The list of signals which are associated with the group. If a particular radix should be used when displaying the signal in <i>nWave</i> , one or more radix specifiers can be included in square brackets ("[]") following each signal name. Multiple radix specifiers can be listed using "," as the delimiter.

An example of a signal interface group for the *AXI* protocol is shown below. Note that signal interface groups are immediate sub-elements of the top *protocol* element. Other details of the protocol definition have been removed or truncated for clarity.

```
<protocol . . . >

    . . .

    <!--AXI Tcl File (Signal Interface Group) Definitions -->
    <signal_if_group>
        if_name="svt_axi_master_if,svt_axi_slave_if"
        name="read_address"
        display_text="Read Address"
        description="Displays the read address signals"
        signals="arvalid, arready, arid, araddr, arlen, arsize, arbust,
                arlock, arcache, arprot, ardomain, arsnoop, arbar,
                read_addr_xact_num[decimal, unsigned]"
    />

    . . .

</protocol>
```

6

Integrating a VIP Using VC Apps Protocol Analyzer

This chapter discusses an example of how to integrate a VIP using *VC Apps Protocol Analyzer* and consists of the following sections:

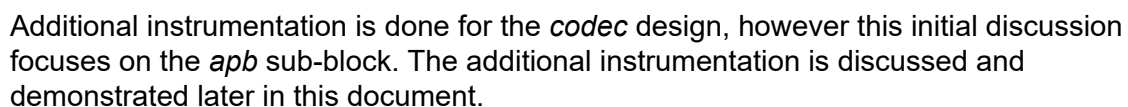
- [Integration Example](#)
- [Integration Steps](#)
- [Creating a Protocol Extension Definition](#)
- [Instrumenting the SystemVerilog Code](#)
- [Modifying the Build or Simulation Environment](#)
- [Modifying the Protocol Analyzer Invocation Environment](#)
- [Configuration Control Considerations](#)
- [Important Rules for Creating Objects](#)

Integration Example

The UVM *codec* example is used in this section to walk through the processing of integrating a VIP using *VC Apps Protocol Analyzer*. The not modified *codec* example code can be found in the following directory:

```
$VERDI_HOME/etc/uvm-1.2/examples/integrated/codec
```

Figure 34 Top-Level Block Diagram for the UVM “codec” Example Design



Integration Steps

This section describes the steps that are required to integrate a VIP using *VC Apps Protocol Analyzer*. The following are the five integration steps:

1. Analysis of the VIP and creation of an initial protocol extension definition

This process includes the following sub-steps:

- Identification of the object type hierarchies that can be captured in the VIP.
- For each object type, identification of the fields that are associated with that type.
- Creation of a *skeleton* protocol extension definition for the VIP that includes the object types and fields identified in the previous two steps.
- Validation of the structure and the contents of the initial protocol extension definition (by setting the `PA_CUSTOM_VIP_PATH` environment variable and examining the protocol definition in the *Protocols* tab of the *Verdi Protocol Analyzer*).

2. Modification of the VIP SystemVerilog code to create a *Protocol Analyzer writer* and capture the objects of interest along with their associated fields (that is, attributes)

This process includes the following sub-steps:

- Analysis of the structure of the VIP and modification to include and import the *VC Apps Protocol Analyzer writer*.
- Analysis of the structure of the VIP and modification to create a *writer* (if requested).
- For each object to be captured, the object must first be created when it starts.
- For each object, each field value must be captured for that object at the appropriate location in the VIP code.
- Each object must be ended at the appropriate location in the VIP code.

3. Modification of the build or the simulation environment to enable successful compilation and simulation of the design

This step primarily requires modification of the SystemVerilog compiler *include path* and specification of the location of the Verdi library.

4. Modification of the environment to enable invocation of the *Verdi Protocol Analyzer*

This step requires modification of the execution environment. This step is also quite simple and primarily requires specification of the location of the Verdi libraries. Also, prior to invoking the *Verdi Protocol Analyzer*, the `PA_CUSTOM_VIP_PATH` environment

variable must be set to point to the location of the required protocol extension definitions.

5. *Tuning of the protocol extension*

This step in the process is the *tuning* of the protocol extension. This process is intended to prompt a general review of the protocol extension definition. This step likely includes modification of the object type colors to make the viewing of simulation results more *pleasing to the eye* (often it is difficult to surmise ahead of time whether colors *clash* when they appear next to each other).

Creating a Protocol Extension Definition

The *apb* block is relatively simple. It has a top-level *agent* that instantiates a monitor sub-block. The monitor sub-block detects *READ* or *WRITE* transactions. Each of these transactions has two fields: a 32-bit address field and a 32-bit data field.

Based on this analysis, the protocol extension definition should include two different object types (to distinguish between the *READ* transactions and *WRITE* transactions). This enables the *Protocol Analyzer* to represent the transactions using different text and colors and also enables the differentiation between these types of transactions when performing searches or examining summary information.

As it is desirable to have these transactions rendered in a single column, the object types for *READ* and *WRITE* transactions should both be sub-types of a top-level transaction object type.

Also, since *READ* and *WRITE* transactions both utilize the *address* and *data* fields, these fields should be associated with this top-level transaction type (as opposed to having copies of these field definitions for the two object sub-types). This results in a *clean* object type definitions and also simplifies searching for all transactions with a particular address or data value.

Firstly, the protocol must be given a name. For this example, the protocol is called *uvm_apb* and is defined in the protocol extension file as follows:

Figure 35 Protocol Extension File

```
<!-- UVM-APB-Protocol-Definition -->
<protocol name="uvm_apb"
  display_text="UVM-APB"
  description="UVM-APB-Protocol-Analyzer-Extension">
```

Next, the top-level transaction object type must be defined. The specification of this object type in the protocol extension file is as follows:

Figure 36 Specification of This Object Type in the Protocol Extension File

```
<!-- Transaction Object Types -->

<!-- Base Transaction Type -->
<object_type name="apb_transaction"
  class_name="apb_rw"
  display_text="Transaction"
  header_text="Transactions"
  background_color="FFFFFF"
  border_color="FF0000"
  font_color="000000"
  icon="icons/apb_transaction.png"
  relative_display_order="1000">

  <description text="The base transaction class which defines the transaction fields." />

  <!-- Transaction Fields -->
  <field name="data"
    data_type="bit.[31:0]"
    display_text="data"
    description="Payload data" />

  <field name="addr"
    data_type="bit.[31:0]"
    display_text="address"
    description="Payload address" />

  <!-- Transaction Sub-Types -->
  <object_sub_type name="apb_read_transaction" />
  <object_sub_type name="apb_write_transaction" />
</object_type>
```

Since this is a top-level object type, it includes the `header_text` attribute. This attribute specifies the text that is shown at the top of the column that displays *READ* and *WRITE* transactions.

Note that this object type definition also includes the definitions for *address* and *data* fields.

And finally, the object type definition specifies the two object sub-types (that are not yet defined) that are used to represent *READ* and *WRITE* transactions.

The specification of *READ* and *WRITE* object types in the protocol extension file is as follows:

The following figure shows the result when this protocol extension definition is applied to sample simulation data:

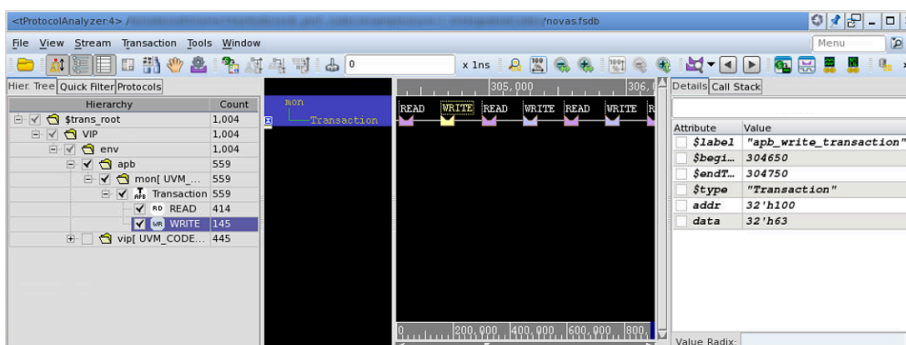
Figure 37 Specification of READ and WRITE Object Types in the Protocol Extension File

```
<!-- Transaction Object Sub-Types -->

<!-- READ Transaction Type -->
<object_type name="apb_read_transaction"
  class_name="apb_rw"
  display_text="READ"
  description="The READ transaction type."
  background_color="C898EF"
  border_color="000000"
  font_color="000000"
  icon="icons/apb_read_transaction.png"
  relative_display_order="1000">
</object_type>

<!-- WRITE Transaction Type -->
<object_type name="apb_write_transaction"
  class_name="apb_wr"
  display_text="WRITE"
  description="The WRITE transaction type."
  background_color="BFBFFF"
  border_color="000000"
  font_color="000000"
  icon="icons/apb_write_transaction.png"
  relative_display_order="2000">
</object_type>
```

Figure 38 Rendering of Sample Simulation Data Using the “uvm_apb” Protocol Extension Definition



The figure above shows the *Hierarchy* pane on the left (which shows the counts for all objects captured during the entire simulation), a zoomed in area with a selected object in the middle (to better see specific *READ* and *WRITE* transactions) and the properties associated with the selected object (which shows the values of *address* and *data* fields).

The complete `uvm_apb` protocol extension definition can be found in the following directory:

```
$VERDI_HOME/share/pa_writer/sv/examples/uvm-1.2/integrated/verdi_pa_extensions/uvm_apb
```

Instrumenting the SystemVerilog Code

This part of the process requires modification of the existing VIP SystemVerilog code. As this is an example that is provided with the UVM distribution, the code for the `apb` block is well-structured.

As previously mentioned, the first step is to include and import the *VC Apps Protocol Analyzer writer*. Among the SystemVerilog code for the `apb` block, there is a file called `apb.sv` that simply includes all of the files required for the block. This file also includes code that creates a package for the block.

The modified version of the `apb.sv` file is shown below.

```
`ifndef APB_SV
`define APB_SV

`include "apb_if.sv"
`include "uvm_macros.svh"
`include "verdi_pa_writer.sv"

package apb_pkg;

  import uvm_pkg::*;
  import verdi_pa_writer_pkg::*;

  typedef virtual apb_if apb_vif;

  typedef class apb_agent;

  `include "apb_rw.sv"
  `include "apb_config.sv"
  `include "apb_master.sv"
  `include "apb_monitor.sv"
  `include "apb_sequencer.sv"
  `include "apb_agent.sv"
endpackage

`endif
```

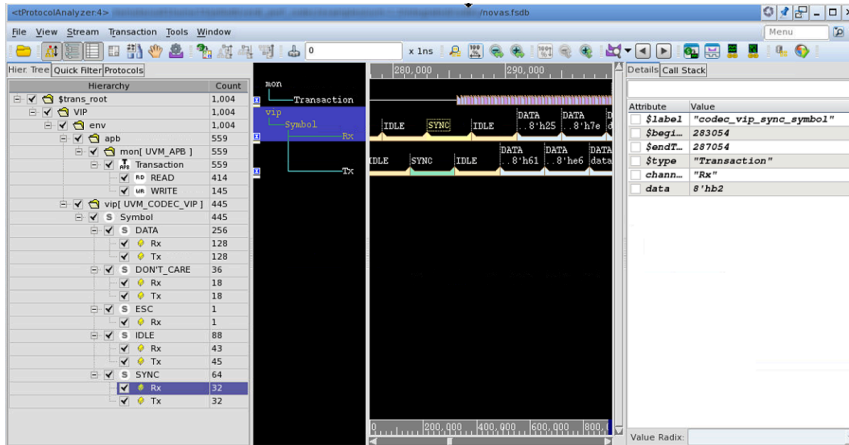
Two lines are added to this file to include and import the *VC Apps Protocol Analyzer writer*. These two lines are highlighted using light red shading.

Note that, in order for this code to compile successfully, the *include* directory path associated with the compilation command must be updated to include the location of the `verdi_pa_writer.sv` file. This topic is explored in more detail in a later section of this document. Also, note that an *import* statement is added to the file, as the classes associated with the *VC Apps Protocol Analyzer writer* are declared within a package called `verdi_pa_writer_pkg`.

The next step involves the creation of a *VC Apps Protocol Analyzer writer* object if requested. The following two things must be taken into account during this step.

- Firstly, any VIP that is modified to utilize *VC Apps Protocol Analyzer* should include the ability to enable or disable object recording because any recording method, no matter how optimized has some impact on performance.
- Secondly, if the VIP may potentially be used in multiple environments, it is best to provide a mechanism for the VIP to be passed an instance of a *VC Apps Protocol Analyzer writer* object from a higher-level component. In fact, the integration of the codec uses this mechanism to have the `apb` objects and other objects associated with a single instance of the `codec`.

A figure of sample data from a simulation of the `codec` is as follows:



As the `apb` UVM agent is the component that is instantiated in a design, the `apb_agent.sv` file must be updated to either create an instance of a *VC Apps Protocol Analyzer writer* object or to store an instance that has been passed to the agent. This requires modifying two parts of the file, the first of which is shown in the following figure:

```
class apb_agent extends uvm_agent;

    apb_sequencer sq;
    apb_master drv;
    apb_monitor mon;

    apb_vif vif;

    verdi_pa_writer pa_writer = null;

    `uvm_component_utils_begin(apb_agent)
        `uvm_field_object(sq, UVM_ALL_ON)
        `uvm_field_object(drv, UVM_ALL_ON)
        `uvm_field_object(mon, UVM_ALL_ON)
    `uvm_component_utils_end

    function new(string name, uvm_component parent = null);
        super.new(name, parent);
    endfunction
endclass
```

One item is added to this file: the declaration of the *VC Apps Protocol Analyzer writer* object. This item is highlighted using light red shading in the figure above. The second modification is made to the *build* phase of the agent, as shown in the following figure:

Chapter 6: Integrating a VIP Using VC Apps Protocol Analyzer Instrumenting the SystemVerilog Code

```

virtual function void build_phase(uvm_phase phase);
  sqr = apb_sequencer::type_id::create("sqr", this);
  drv = apb_master::type_id::create("drv", this);
  mon = apb_monitor::type_id::create("mon", this);

  if (!uvm_config_db#(apb_vif)::get(this, "", "vif", vif)) begin
    `uvm_fatal("APB/AGT/NOVIF", "No virtual interface specified for this agent instance")
  end

  // Check if the Verdi Protocol Analyzer writer is null.
  // If it is null, check if recording is enabled in the configuration database.
  // If recording is enabled in the database, create the writer for the "uvm_apb" protocol.
  if (pa_writer == null) begin
    bit enable_pa_writer = 0;

    if (uvm_config_db#(bit)::get(this, "", "enable_pa_writer", enable_pa_writer)) begin
      if (enable_pa_writer == 1) begin
        string apb_if_path;

        pa_writer = new( { get_full_name(), ".mon" }, "uvm_apb", "1.0" );

        // If an interface path has been specified in the configuration database,
        // associate that path with the Protocol Analyzer writer.
        if (uvm_config_db#(string)::get(this, "", "apb_if_path", apb_if_path)) begin
          pa_writer.add_if_paths( apb_if_path );
        end
      end
    end
  end

  // If the writer is not null, pass it to the monitor.
  if (pa_writer != null) begin
    mon.set_pa_writer( pa_writer );
  end
endfunction: build_phase

```

Although more code is modified in this section of the file, the code is quite straightforward.

The purpose of this logic is to determine whether or not a writer needs to be created *locally*.

The code includes two conditional (that is, if) statements. The first conditional statement checks if the writer is null.

If the writer is null, the code checks the local context of the configuration database for the `enable_pa_writer` bit attribute. If this attribute is found and is set to 1, a local instance of the writer is created for version 1.0 of the `uvm_apb` protocol. The instance path that is passed to the writer's constructor is created by taking the path to the agent and appending `.mon` (to indicate that the monitor component is collecting data; however, whether or not `.mon` should be appended to the instance path is purely an implementation decision).

The final lines of code in this first conditional block checks the local context of the configuration database for the `apb_if_path` attribute. If this attribute is found, it is added to the list of interface paths for this instance of the VIP.

There are a number of different ways that the interface path can be passed to the VIP.

Firstly, note that the interface path is different from the VIP hierarchy path. The interface path comes from the location where the interface is instantiated in the design, which is often at the top level. The configuration database is a handy mechanism for capturing that information in another block in the design hierarchy, but have that information be available to instances of the VIP using a general, reusable approach. In this example, the name of the attribute (`apb_if_path`) is arbitrarily chosen. The important point is that this attribute is added to the configuration database in the code where the interface is instantiated (more

on this later), so that the instance of the VIP can retrieve that path (and add it to the writer) no matter where the VIP ended up being instantiated in the test environment hierarchy.

The second conditional statement also checks if the writer is null. If the writer is not null, it is passed to the instance of the `apb_monitor` component by calling the `set_pa_writer` method for the monitor class.

The monitor component includes the code that actually detects *READ* and *WRITE* transactions. Thus, it is the appropriate class for adding the code that utilizes the *VC Apps Protocol Analyzer writer* object to record protocol activity.

In a manner almost identical to the agent class, the `apb_monitor.sv` file must be updated to store an instance that is passed to the monitor from the agent. In addition, the monitor class must be updated to record *READ* and *WRITE* transactions (along with their associated attributes). This requires modifying two parts of the file, the first of which is shown in the following figure:

The following two items are added to this file:

- One item declares the VC Apps Protocol Analyzer writer object.
- The other item defines a function for setting the writer instance.

These two items have been highlighted using light red shading in the following figure:

```
class apb_monitor extends uvm_monitor;
  virtual apb_if passive sigs;

  uvm_analysis_port#(apb_rw) ap;
  apb_config cfg;
  verdi_pa_writer pa_writer = null;

  `uvm_component_utils(apb_monitor)

function new(string name, uvm_component parent = null);
  super.new(name, parent);
  ap = new("ap", this);
endfunction

virtual function void set_pa_writer( verdi_pa_writer pa_writer );
  this.pa_writer = pa_writer;
endfunction
```

The second set of modifications are done to the `run` phase of the monitor, as shown in the following figure:

```

virtual task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    apb_rw tr;
    string pa_uid;
    string pa_object_type;

    // Wait for a SETUP cycle
    do begin
      @(this.sigs.pok);
    end
    while (this.sigs.pok.psel != 1'b1 || this.sigs.pok.penable != 1'b0);

    tr = apb_rw::type_id::create("tr", this);
    tr.kind = (this.sigs.pok.pwrite) ? apb_rw::WRITE : apb_rw::READ;
    tr.addr = this.sigs.pok.paddr;

    if (pa_writer != null) begin
      pa_object_type = (tr.kind == apb_rw::READ) ? "apb_read_transaction" : "apb_write_transaction";
      pa_uid = pa_writer.create_object(pa_object_type);

      if (pa_uid != "") begin
        uvm_info("APB/MON/PA", $formatf("Object started (uid=%0s, kind=%0s)", pa_uid, tr.kind), UVM_HIGH);
        void'(pa_writer.set_object_attribute_value_bit_vector(pa_uid, "addr", tr.addr, 32));
        uvm_info("APB/MON/PA", $formatf("Object updated (uid=%0s, addr=%0h)", pa_uid, tr.addr), UVM_HIGH);
      end
    end

    @(this.sigs.pok);
    if (this.sigs.pok.penable != 1'b1) begin
      uvm_error("APB", "APB protocol violation: SETUP cycle not followed by ENABLE cycle");
    end
    tr.data = (tr.kind == apb_rw::READ) ? this.sigs.pok.pdata : this.sigs.pok.pwdata;

    if ((pa_writer != null) && (pa_uid != "")) begin
      void'(pa_writer.set_object_attribute_value_bit_vector(pa_uid, "data", tr.data, 32));
      uvm_info("APB/MON/PA", $formatf("Object updated (uid=%0s, data=%0h)", pa_uid, tr.data), UVM_HIGH);
    end

    trans_observed(tr);
    uvm_do_callbacks(apb_monitor, apb_monitor_obs, trans_observed(this, tr));

    if ((pa_writer != null) && (pa_uid != "")) begin
      void'(pa_writer.end_object(pa_uid));
      uvm_info("APB/MON/PA", $formatf("Object ended (uid=%0s)", pa_uid), UVM_HIGH);
    end

    apb.write(tr);
  end
endtask: run_phase

```

There are three basic steps that are performed in the modified code when an object is recorded:

- If the writer is not null, create an object using the appropriate object type name, the channel name (if applicable), and the start time for the object (if earlier than the current time)
- If the object is successfully created, add attributes to the object when their values are known (this code is repeated for each attribute to be recorded)
- When the end time of the object is reached (assuming that the object was initially created successfully), end the object.

Depending upon how many types of objects are being captured (and the structure of the VIP code), this general object processing pattern may be repeated in the code multiple times. However, in this example, both *READ* and *WRITE* transactions are detected in one location, so this pattern appears only once. The modifications to the code are highlighted using light red shading.

The first major block of code first checks if objects should be recorded (that is, if the writer is not null). If so, the object type name is set according to the APB transaction type. Note that the two possible values (*apb_read_transaction* and *apb_write_transaction*) match the names of object types defined in the protocol extension.

Using this name, a new object is created by calling the *create_object* method of the writer class⁴. This method returns an object *uid* (unique identifier), which is a string that is used

4. For more information about VC Apps Protocol Analyzer writer methods, refer to the appendix that lists and describes all of the methods available in the VC Apps Protocol Analyzer writer API.

to reference the object in later operations. If an empty string is returned, there is an error creating the object (in which case the error is reported in the simulation log).

If the object is successfully created, the *addr* attribute is set for the object by calling the *set_object_attribute_bit_vector* method of the writer class. This attribute is set at this point in the code, as the address is known and fixed. In general, it is recommended that attributes be set when their values are known and fixed (as opposed to setting the values of all attributes at the time an object ends). Using this approach, if an unexpected error stops the simulation before it can complete *cleanly*, those attribute values that are recorded are available to potentially help track down the source of the error.

The second major block of code sets the *data* attribute for the object. Finally, the third major block of code ends the object.

There is one topic that is not yet described: capturing interface paths and adding the paths to the configuration database.

This operation is performed in the *tb_top.sv* file in the *codec* sub-directory. This file contains the code that instantiates the interfaces.

```
module tb_top;

  reg clk, sclk;
  wire rst;

  wire tx, rx;
  wire intr;

  tb_ctl_if ctl(clk, sclk, rst, intr);
  apb_if apb0(clk);
  vip_if vip0(sclk, rx, tx);

  dut dut0(tx, rx, sclk, apb0, intr, clk, rst);

  initial
  begin
    uvm_config_db #(string)::set( null, "*", "apb_if_path", "tb_top.apb0" );
    uvm_config_db #(string)::set( null, "*", "vip_if_path", "tb_top.vip0" );
  end
endmodule
```

Two lines of code are added to this file:

1. The first line specifies the hierarchical design path for the APB interface
2. The second line specifies the hierarchical design path for the (codec) “VIP” interface. Since the name of the top-level module is *tb_top* and the interfaces are directly instantiated within this module, the paths to these two interfaces are known and can be directly added to the configuration database.

This is a simple example of adding the paths to the database; other mechanisms may be available to the verification engineer, based on the overall structure of the verification environment. For example, if the test environment utilizes a general configuration object that already contains handles to the interfaces, the underlying VIP code could extract the configuration object and get the path to the interface directly.

Modifying the Build or Simulation Environment

Modification of the build or the simulation environment is quite simple. The modified VCS *makefile* is shown in the following figure

```
UVM_HOME      = ../../..
VERDI_TAB      = ${NOVAS_HOME}/share/PLI/VCS/${PLATFORM}/novas.tab
VERDI_PLI      = ${NOVAS_HOME}/share/PLI/VCS/${PLATFORM}/pli.a

include ../../Makefile.vcs

all: comp run

comp:
    $(VCS) $(COPTS) +incdir+./apb+vip+../../src \
    -P ${VERDI_TAB} ${VERDI_PLI} \
    tb_top.sv test.sv

run: test hw_reset_test

test:
    $(SIMV) $(ROPTS)
    $(CHECK)

hw_reset_test:
    $(SIMV) $(ROPTS) +UVM_TESTNAME=hw_reset_test
    $(CHECK)
```

The changes (highlighted using light red shading) modifies the SystemVerilog compiler *include path* and specifies the location of the Verdi library.

A detailed discussion of linking the Verdi libraries with VCS and other simulators can be found in the following file:

`$VERDI_HOME/doc/linking_dumping.pdf`

Modifying the Protocol Analyzer Invocation Environment

Before invoking the *Verdi Protocol Analyzer*, the `PA_CUSTOM_VIP_PATH` environment variable must be set to point to the location of custom protocol extension definitions.

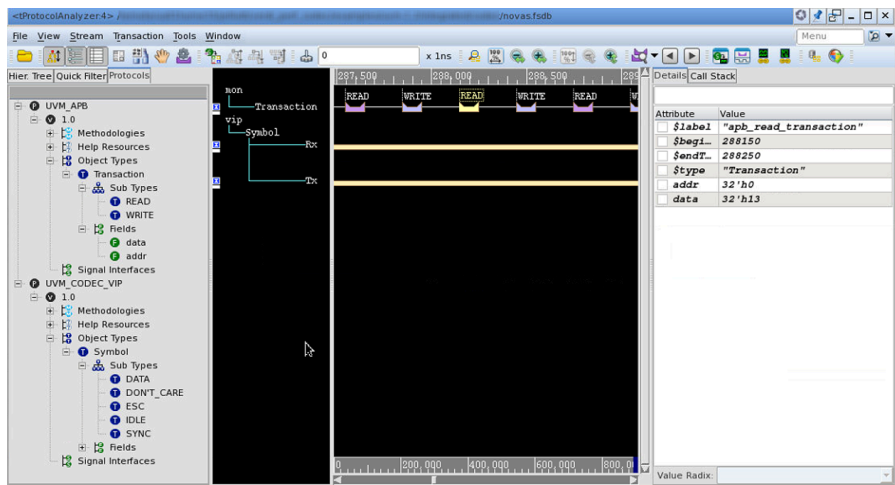
This can be accomplished for this example with the following command:

```
setenv PA_CUSTOM_VIP_PATH
$VERDI_HOME/share/pa_writer/sv/examples/uvm-1.2/integrated/verdi_pa_extensions
```

Note that multiple directories can be specified each separated by a colon (':').

After setting this environment variable, any custom protocol extension definitions can be validated by examining the *Protocols* tab in the *Protocol Analyzer*.

Figure 39 Protocols Tab Showing the "UVM APB" Protocol Extension Definition



Configuration Control Considerations

In the `apb` example, there is a single configuration database attribute that controls whether or not to enable the recording of objects. As this component only records one kind of activity, the use of a single attribute to control recording is appropriate. However, for some VIPs, it is useful to provide a more granular set of controls to determine what kind of data is recorded.

Some items to consider are as follows:

- The ability to enable or disable object recording (global).
- The ability to enable or disable object capture based on object types, protocol layers, and so on.
- The ability to control the amount of data that is captured.
- The ability to make mid-simulation configuration changes.

As previously mentioned, any recording method, no matter how optimized has some impact on simulation performance. In general, the level of impact is difficult to quantify, as it is highly dependent on the complexity of the protocol, the capabilities of the VIP, and the extent of instrumentation (that is, how many objects are recorded and the number and type of attributes that are recorded for each object).

As an example, the following table below shows the recording controls for the *Synopsys USB 3.0 VIP*.

Variable	Value	Meaning
enable_prot_pa_gen	0	No PA objects should be generated for protocol-layer activity.
	1	PA objects should be generated for transfers only
	2	PA objects should be generated for transfers and transactions
enable_link_pa_gen	0	No PA objects should be generated for link-layer activity
	1	PA objects should be generated for packets only
	2	PA objects should be generated for packets and link commands
	3	PA objects should be generated for packets, link commands and symbol sets
	4	PA objects should be generated for packets, link commands, symbol sets and LTSSM states
enable_phys_pa_gen	0	No PA objects should be generated for physical-layer activity
	1	PA objects should be generated for LFPS signaling
enable_payload_pa_gen	0	No payload data should be written out for packets, transactions and transfers
	1	Payload data should be written out for packets, transactions and transfers

Note that these variables control object recording not only by layer (physical, link, and protocol) but for each of those layers, the value of the variable controls the amount of data that is recorded. Note that there is a separate variable that controls whether or not payload data is recorded, as the size of the payload data can be significant.

Important Rules for Creating Objects

When an object is created using the `create_object` method of the *VC Apps Protocol Analyzer API*, the object type must be specified. However, in some VIPs under certain conditions, it is not possible to know the type of the object at the time the transaction is intended to start. Sometimes it takes one or more additional clock cycles to determine the object type, especially if the protocol extension defines a complex object type hierarchy.

The `create_object` method supports this situation, by providing an optional object start time that is earlier than the current simulation time.

Note:

The use of this feature, while occasionally necessary, is highly discouraged. Before utilizing this capability, a thorough analysis of the VIP should be performed to determine if it is possible to remedy this situation by making changes to the VIP code. If this is not possible, it is imperative to ensure that the rules described in this section are never violated.

Even if these rules are carefully followed, the use of this feature introduces some impact on performance.

The following diagram shows three different scenarios which help illustrate this rule. These scenarios show the creation of two objects (labeled *A* and *B*) at different simulation times.

A

Appendix A- VC Apps Protocol Analyzer SystemVerilog Writer API

This appendix discusses about the VC Apps Protocol Analyzer SystemVerilog Writer APIs and consists of the following sections:

- [Writer Macros or Defines](#)
 - [ENABLE_VERDI_PA_WRITER](#)
 - [VERDI_PA_WRITER_BV_MAX_WIDTH](#)
- [General Writer Methods](#)
 - [new](#) (constructor)
 - [flush_writer](#)
- [Interface Methods](#)
 - [add_if_paths](#)
- [General Object Methods](#)
 - [create_object](#)
 - [end_object](#)
- [Object Relationship Methods](#)
 - [set_object_parent](#)
 - [add_object_child](#)
 - [add_object_children](#)
 - [set_object_predecessor](#)
 - [add_object_successor](#)
 - [add_object_successors](#)
- [Object Attribute Methods](#)

- [set_object_attribute_value_bit](#)
- [set_object_attribute_value_bit_vector](#)
- [set_object_attribute_value_int](#)
- [set_object_attribute_value_real](#)
- [set_object_attribute_value_time](#)
- [set_object_attribute_value_string](#)
- **Memory Methods**
 - [mem_init](#)
 - [mem_init_range](#)
 - [mem_read](#)
 - [mem_write](#)
 - [mem_write_masked](#)
 - [mem_peek](#)
 - [mem_poke](#)
 - [mem_load_file](#)
 - [mem_file_data](#)
 - [mem_unload](#)
 - [mem_unload_all](#)

Writer Macros or Defines

ENABLE_VERDI_PA_WRITER

This directive must be defined in order to enable the functionality defined in the *VC Apps Protocol Analyzer writer* class. The intent of this macro is to allow the compilation and use of the VIP outside of the Verdi environment. By default, this macro is not defined.

VERDI_PA_WRITER_BV_MAX_WIDTH

This macro defines the maximum width of a bit-vector that can be written to the FSDB. If this macro is not explicitly defined by the user, the value for this macro is set to 1024.

See the `set_object_attribute_value_bit_vector` method for additional details.

General Writer Methods

new (constructor)

This method constructs a new instance of the `verdi_pa_writer` class. If the writer needs to be used to capture protocol-level transactions, then the protocol name and version should be specified. If the writer needs to be used to capture memory actions, then the address width, data width, and access type should be specified.

Parameters

Name	Description
<code>instance_path</code>	The path of the instance with which the writer is associated.
<code>protocol_name</code>	The name of the protocol with which the objects being written are associated.
<code>protocol_version</code>	The version of the protocol.
<code>addr_width</code>	The memory address width (number of bits), or -1 if the writer is only used for capturing protocol-level transactions.
<code>data_width</code>	The memory data width (number of bits), or -1 if the writer is only used for capturing protocol-level transactions.
<code>access_type</code>	The memory access type (<code>READ_WRITE</code> or <code>READ_ONLY</code>). This value is only meaningful if the writer is used for capturing memory actions.

Prototype

```
function new( string      instance_path,  
             string      protocol_name  = "",  
             string      protocol_version = "",  
             int         addr_width     = -1,  
             int         data_width     = -1,  
             access_type_enum access_type = READ_WRITE );
```

flush_writer

This method ends any un-ended objects associated with this instance of the `verdi_pa_writer` class.

Parameters

None

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit flush_writer();
```

Interface Methods

add_if_paths

This method adds the specified hierarchical design path to the list of interface paths associated with the design for the protocol with which the writer is associated.

Parameters

Name	Description
if_path	The hierarchical design path of the interface to be added.

Prototype

```
function string add_if_paths( string if_path );
```

General Object Methods

create_object

This method creates a new protocol object and returns the `uid` for the newly-created object.

Parameters

Name	Description
object_type	The type of object to be created.
object_uid	The <code>uid</code> for the object to be created. If not specified, a <code>uid</code> is automatically created, based on the specified object type.
object_channel	The name of the channel with which the object is to be associated. If an empty string is specified, the object is not associated with any channel (which is the default condition for an object).
start_time	The start time of the object. If the start time is not specified, the current simulation time is set as the start time.

Return Value

This method returns the `uid` of the new object. If the object `uid` was specified, the same string is returned. An empty string indicates that an error occurred while attempting to create the new object.

Prototype

```
function string create_object( string object_type,  
                             string object_uid = "",  
                             string object_channel = "",  
                             time    start_time = -1 );
```

end_object

This method ends a protocol object. When this method is called, the end time of the object is set to the current simulation time. At this point, all information about the object is considered to be specified. Thus, no further changes can be made to the attributes associated with the object.

It is important that all objects be ended at an appropriate time during the simulation.

Parameters

Name	Description
object_uid	The <code>uid</code> of the object to be ended

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit end_object( string object_uid );
```

Object Relationship Methods

set_object_parent

This method specifies the `uid` of the parent object for the object with the specified `uid`. This method can be called up until the point at which the object is ended.

At the time this method is called, no checks are performed to validate the `uid` that is specified for the parent object. This allows a parent object to be specified as follows:

- An object that is not yet created (assuming that object `uids` are being managed or constructed by the VIP and are not being automatically generated by the VIP writer).

Or

- The parent object is created (so that the `uid` of the object is constructed), but that the object has not yet begun.

If at the time the simulation ends, no object with the `uid` specified for the parent object is created, PA reports this situation when the data created by the VIP is being read into a protocol view in a project.

Parameters

Name	Description
<code>object_uid</code>	The <code>uid</code> of the object whose parent object is to be specified
<code>parent_object_uid</code>	The <code>uid</code> of the parent object.

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_parent( string object_uid,  
                               string parent_object_uid );
```

add_object_child

This method adds a child object to the object with the specified `uid`. An object can have multiple child objects, as appropriate for the protocol. This method can be called up until the point at which the object is ended.

At the time this method is called, no checks are performed to validate the `uid` that is specified for the child object. This allows a child object to be specified as follows:

- An object that is not yet created (assuming that object `uids` are being managed or constructed by the VIP and are not being automatically generated by the VIP writer).

Or

- The child object is created (so that the `uid` of the object is constructed), but that the object has not yet begun.

If at the time the simulation ends, no object with the `uid` specified for the child object is created, PA reports this situation when the data created by the VIP is being read into a protocol view in a project.

Parameters

Name	Description
<code>object_uid</code>	The <code>uid</code> of the object to which a child object is to be added
<code>child_object_uid</code>	The <code>uid</code> of the child object

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit add_object_child( string object_uid,  
                             string child_object_uid );
```

add_object_children

This method adds an array of child objects to the object with the specified `uid`. This method can be called up until the point at which the object is ended.

At the time this method is called, no checks are performed to validate the `uids` that are specified for the child objects. This allows any or all of the child objects to be specified as follows:

- Objects that are not yet created (assuming that object `uids` are being managed or constructed by the VIP and are not being automatically generated by the VIP writer).

Or

- The child objects are created (so that the `uids` of the objects are constructed), but that the object has not yet begun.

If at the time the simulation ends, no object with the `uid` specified for any of the child objects is created, PA reports this situation when the data created by the VIP is being read into a protocol view in a project.

Parameters

Name	Description
<code>object_uid</code>	The <code>uid</code> of the object to which the child objects are to be added
<code>child_object_uids</code>	The <code>uids</code> of the child objects

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit add_object_children( string object_uid,  
                                string child_object_uids[] );
```

set_object_predecessor

This method specifies the `uid` of the predecessor object for the object with the specified `uid`. This method can be called up until the point at which the object is ended.

At the time this method is called, no checks are performed to validate the `uid` that is specified for the predecessor object. This allows a predecessor object to be specified as follows:

- An object that is not yet created (assuming that object `uids` are being managed or constructed by the VIP and are not being automatically generated by the VIP writer).

Or

- The predecessor object is created (so that the `uid` of the object is constructed), but that the object has not yet begun.

If at the time the simulation ends, no object with the `uid` specified for the predecessor object is created, PA reports this situation when the data created by the VIP is being read into a protocol view in a project.

Parameters

Name	Description
<code>object_uid</code>	The <code>uid</code> of the object whose predecessor object is to be specified
<code>predecessor_object_uid</code>	The <code>uid</code> of the predecessor object
<code>predecessor_writer</code>	The <code>verdi_pa_writer</code> writer with which the predecessor object is associated or null for the current writer

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_predecessor(  
    string object_uid,  
    string predecessor_object_uid,  
    verdi_pa_writer predecessor_writer = null );
```

add_object_successor

This method adds a successor object to the object with the specified `uid`. An object can have multiple successor objects, as appropriate for the protocol. This method can be called up until the point at which the object is ended.

At the time this method is called, no checks are performed to validate the `uid` that is specified for the successor object. This allows a successor object to be specified as follows:

- An object is not yet created (assuming that object `uids` are being managed or constructed by the VIP and are not being automatically generated by the VIP writer).

Or

- The successor object is created (so that the `uid` of the object is constructed), but that the object has not yet begun.

If at the time the simulation ends, no object with the `uid` specified for the successor object is created, PA reports this situation when the data created by the VIP is being read into a protocol view in a project.

Parameters

Name	Description
<code>object_uid</code>	The <code>uid</code> of the object to which a successor object is to be added
<code>successor_object_uid</code>	The <code>uid</code> of the successor object.
<code>successor_writer</code>	The <code>verdi_pa_writer</code> writer with which the successor object is associated or null for the current writer

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit add_object_successor(  
    string object_uid,  
    string successor_object_uid,  
    verdi_pa_writer successor_writer = null );
```

add_object_successors

This method adds an array of successor objects to the object with the specified `uid`. This method can be called up until the point at which the object is ended.

At the time this method is called, no checks are performed to validate the `uids` that are specified for the successor objects. This allows any or all of the successor objects to be specified as follows:

- Objects that have are not yet created (assuming that object `uids` are being managed or constructed by the VIP and are not being automatically generated by the VIP writer).
- Or
- The successor objects are created (so that the `uids` of the objects are constructed), but that the object has not yet begun.

If at the time the simulation ends, no object with the `uid` specified for any of the successor objects is created, PA reports this situation when the data created by the VIP is being read into a protocol view in a project.

Parameters

Name	Description
<code>object_uid</code>	The <code>uid</code> of the object to which the successor objects are to be added
<code>successor_object_uids</code>	The <code>uids</code> of the successor objects
<code>successor_writer</code>	The “ <code>verdi_pa_writer</code> ” writer with which the successor objects are associated or null for the current writer

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit add_object_successors(  
    string object_uid,  
    string successor_object_uids[],  
    verdi_pa_writer successor_writer = null );
```

Object Attribute Methods

set_object_attribute_value_bit

This method specifies the value of a bit attribute for an object. This method can be called up until the point at which the object is ended. However, only the last value specified is associated with the attribute.

Parameters

Name	Description
object_uid	The <code>uid</code> of the object to be modified
attribute_name	The name of the attribute whose value is being specified
attribute_value	The attribute value
expected_value	The expected attribute value. If this value differs from <code>attribute_value</code> , the object is marked as having an error condition
has_expected	A flag that indicates whether or not an expected value is specified

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_attribute_value_bit ( string object_uid,  
                                             string attribute_name,  
                                             bit   attribute_value,  
                                             bit   expected_value = 0,  
                                             bit   has_expected = 0 );
```

set_object_attribute_value_bit_vector

This method specifies the value of a bit-vector attribute for an object. This method can be called up until the point at which the object is ended. However, only the last value specified is associated with the attribute.

Parameters

Name	Description
object_uid	The uid of the object to be modified
attribute_name	The name of the attribute whose value is being specified
attribute_value	The attribute value
num_bits	The size of the bit-vector. For example, if the definition of the bit-vector to be stored is <code>bit [15:0] data</code> , the value of this parameter should be set to 16. If the value of this parameter exceeds <code>VERDI_PA_WRITER_BV_MAX_WIDTH</code> , the stored value is pruned (and thus data can be lost)
expected_value	The expected attribute value. If this value differs from <code>attribute_value</code> , the object is marked as having an error condition
has_expected	A flag that indicates whether or not an expected value is specified

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_attribute_value_bit_vector(
    string      object_uid,
    string      attribute_name,
    bit [ `VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] attribute_value,
    int         num_bits = `VERDI_PA_WRITER_BV_MAX_WIDTH,
    bit [ `VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] expected_value = 0,
    bit         has_expected = 0 );
```

set_object_attribute_value_int

This method specifies the value of an integer attribute for an object. This method can be called until the point at which the object is ended. However, only the last value specified is associated with the attribute.

Parameters

Name	Description
object_uid	The <code>uid</code> of the object to be modified
attribute_name	The name of the attribute whose value is being specified
attribute_value	The attribute value
num_bits	The number of bits required to store the integer. Legal values are 16, 32 or 64 for storing shortint, integer or longint values, respectively
expected_value	The expected attribute value. If this value differs from <code>attribute_value</code> , the object is marked as having an error condition
has_expected	A flag that indicates whether or not an expected value is specified

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_attribute_value_int string  object_uid,  
                                           string  attribute_name,  
                                           longint attribute_value,  
                                           int      num_bits = 32,  
                                           longint expected_value = 0,  
                                           bit      has_expected = 0 );
```

set_object_attribute_value_real

This method specifies the value of a real attribute for an object. This method can be called until the point at which the object is ended. However, only the last value specified is associated with the attribute.

Parameters

Name	Description
object_uid	The <code>uid</code> of the object to be modified.

Name	Description
attribute_name	The name of the attribute whose value is being specified
attribute_value	The attribute value
expected_value	The expected attribute value. If this value differs from <code>attribute_value</code> , the object is marked as having an error condition
has_expected	A flag that indicates whether or not an expected value is specified

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_attribute_value_real( string object_uid,
                                           string attribute_name,
                                           real attribute_value,
                                           real expected_value = 0,
                                           bit has_expected = 0 );
```

set_object_attribute_value_time

This method specifies the value of a time attribute for an object. This method can be called up until the point at which the object is ended. However, only the last value specified is associated with the attribute.

Parameters

Name	Description
object_uid	The <code>uid</code> of the object to be modified
attribute_name	The name of the attribute whose value is being specified
attribute_value	The attribute value
expected_value	The expected attribute value. If this value differs from <code>attribute_value</code> , the object is marked as having an error condition

Name	Description
has_expected	A flag that indicates whether or not an expected value is specified

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_attribute_value_time( string object_uid,  
                                             string attribute_name,  
                                             time   attribute_value,  
                                             time   expected_value = 0,  
                                             bit    has_expected = 0 );
```

set_object_attribute_value_string

This method specifies the value of a string attribute for an object. This method can be called until the point at which the object is ended. However, only the last value specified is associated with the attribute.

Parameters

Name	Description
object_uid	The <code>uid</code> of the object to be modified
attribute_name	The name of the attribute whose value is being specified
attribute_value	The attribute value
expected_value	The expected attribute value. If this value differs from <code>attribute_value</code> , the object is marked as having an error condition
has_expected	A flag that indicates whether or not an expected value is specified

Return Value

The status of the operation; 1 = success, 0 = failure.

Prototype

```
function bit set_object_attribute_value_string(
    string object_uid,
    string attribute_name,
    string attribute_value,
    string expected_value = "",
    bit has_expected = 0 );
```

Memory Methods

mem_init

This method initializes the entire memory by creating a `create_pattern` memory action at the current time. The memory is initialized with a virtual pattern, which means that calling this method to initialize the memory does not result in the actual allocation of simulator memory (that is, data values are calculated *on-the-fly*).

Parameters

Name	Description
pattern	The pattern to use to initialize the entire memory (CONST, INCR, DECR, WALK_LEFT, WALK_RIGHT, or RAND)
base_data	The base data to use for memory initialization. That is, the value depends on the specified pattern

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_init( pattern_type_enum pattern,
    longint base_data );
```

mem_init_range

This method initializes the specified address range by creating a `create_pattern` memory action at the current time. The specified address range is initialized with a virtual pattern, which means that calling this method to initialize the memory does not result in the actual allocation of simulator memory (that is, data values are calculated *on-the-fly*).

Parameters

Name	Description
pattern	The pattern to use to initialize the specified address range (CONST, INCR, DECR, WALK_LEFT, WALK_RIGHT, or RAND)
base_data	The base data to use for memory initialization. That is, the value depends on the specified pattern
start_addr	The start address for the address range to be initialized
end_addr	The end address for the address range to be initialized

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_init_range( pattern_type_enum pattern,  
                             longint          base_data,  
                             longint unsigned start_addr,  
                             longint unsigned end_addr );
```

mem_read

This method creates a `read` memory action at the current time.

Parameters

Name	Description
addr	The address being read
data	The data that was read

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_read( longint unsigned addr,  
                      bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data );
```

mem_write

This method creates a `write` memory action at the current time.

Parameters

Name	Description
addr	The address being written
data	The data that was written

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_write( longint unsigned addr,  
                        bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data );
```

mem_write_masked

This method creates a `write_masked` memory action at the current time.

Parameters

Name	Description
addr	The address being written
data	The data that is written
data_mask	The mask that is applied to the data being written
data_result	The final data for the address after applying the mask and the previous data value

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit  
    mem_write_masked( longint unsigned addr,
```

```
bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data,  
bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data_mask,  
bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data_result );
```

mem_peek

This method creates a `peek` memory action at the current time.

Parameters

Name	Description
addr	The address being peeked
data	The data that is peeked

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_peek( longint unsigned addr,  
                      bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data );
```

mem_poke

This method creates a `poke` memory action at the current time.

Parameters

Name	Description
addr	The address being poked
data	The data that was poked

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_poke( longint unsigned addr,  
                      bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data );
```

mem_load_file

This method creates a `load_file` memory action at the current time.

Parameters

Name	Description
<code>file_path</code>	The path of the file that was read

Return Value

This method returns the `uid` of the new object. An empty string indicates that an error occurred while attempting to create the new object.

Prototype

```
string mem_load_file( string file_path );
```

mem_file_data

This method creates a `file_data` memory action at the current time.

Parameters

Name	Description
<code>load_file_object_uid</code>	The <code>uid</code> of the <code>load_file</code> memory action with which this file data is associated. A parent-child relationship is created between the specified <code>load_file</code> transaction and the newly-created <code>file_data</code> transaction
<code>addr</code>	The address with which the file memory value is associated
<code>data</code>	The data with which the file memory value is associated

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_file_data( string load_file_object_uid,  
                           longint unsigned addr,  
                           bit [`VERDI_PA_WRITER_BV_MAX_WIDTH - 1:0] data );
```

mem_unload

This method unloads the specified address range by creating an `unload` memory action at the current time.

Parameters

Name	Description
<code>start_addr</code>	The start address for the address range to be unloaded
<code>end_addr</code>	The end address for the address range to be unloaded

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_unload( longint unsigned start_addr,  
                        longint unsigned end_addr );
```

mem_unload_all

This method unloads the entire memory by creating an `unload` memory action at the current time.

Return Value

This method returns the status of the operation (true = success; false = failure).

Prototype

```
function bit mem_unload_all();
```